



Project: Heart Disease Diagnosis with Tree-based Machine Learning Algorithms

Dương Đình Thắng

Dương Trường Bình

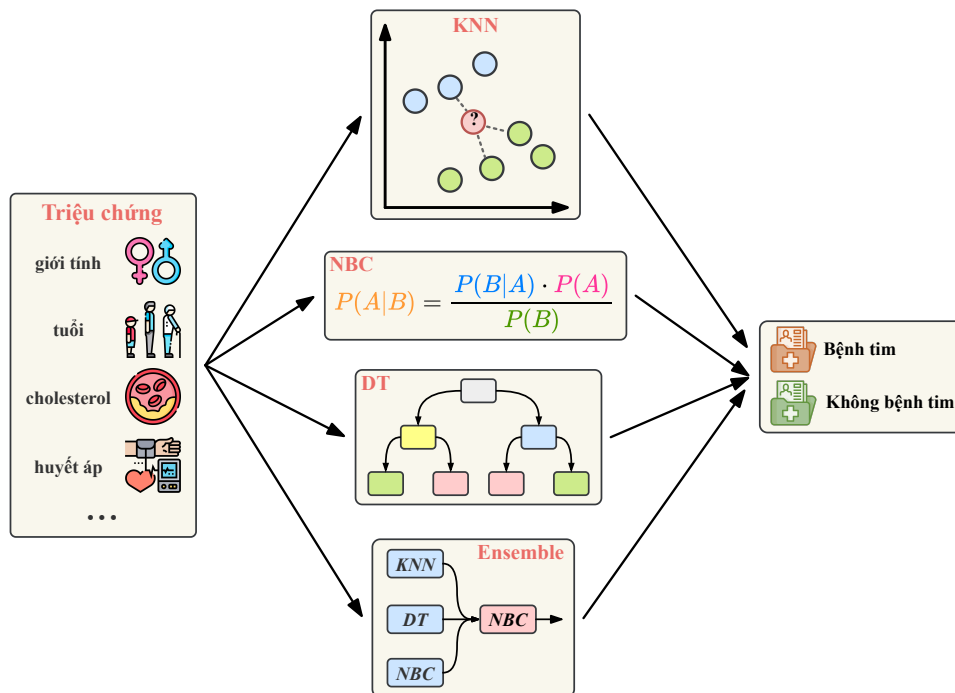
Đình Quang Vinh

1. Giới thiệu

1.1. Bối cảnh

Bệnh tim mạch hiện là nguyên nhân gây tử vong hàng đầu trên thế giới, với khoảng 20,5 triệu ca vào năm 2021, chiếm xấp xỉ 32% tổng số ca tử vong toàn cầu [1–3]. Tại Việt Nam, ước tính của WHO năm 2016 cho thấy 31% trường hợp tử vong có liên quan đến bệnh tim mạch. Các dự báo gần đây cũng chỉ ra rằng 80% gánh nặng tử vong do bệnh tim sẽ rơi vào các quốc gia có thu nhập từ thấp đến trung bình, nơi việc tầm soát và điều trị còn gặp nhiều hạn chế về chi phí và nguồn lực [3, 4].

Trước thực trạng đó, các giải pháp hỗ trợ chẩn đoán sớm và chính xác đóng vai trò quan trọng. Trong những năm gần đây, các mô hình machine learning đã nổi lên như một hướng tiếp cận hiệu quả, nhờ khả năng xử lý khối lượng dữ liệu lớn và phát hiện các mối quan hệ ẩn giữa các đặc trưng y tế.



Hình 1: Tổng quan dự án chẩn đoán bệnh tim bằng các phương pháp machine learning.

Mục lục

1.	Giới thiệu	1
1.1.	Bối cảnh	1
1.2.	Mục tiêu và phạm vi	4
2.	Bộ dữ liệu và tiền xử lý	4
2.1.	Bộ dữ liệu Cleveland Heart Disease	4
2.2.	Khám phá dữ liệu	6
2.3.	Xây dựng bốn biến thể dữ liệu	9
3.	Benchmark các mô hình Machine Learning	17
3.1.	Khung thực nghiệm chung	17
3.2.	Baseline không thuộc họ cây: Naive Bayes và K-Nearest Neighbors	20
3.3.	Decision Tree	22
3.4.	Random Forest	24
3.5.	AdaBoost	27
3.6.	Gradient Boosting	29
3.7.	XGBoost	31
4.	Giải thích mô hình bằng TreeSHAP	33
4.1.	Động lực	33
4.2.	TreeSHAP	33
4.3.	Triển khai TreeSHAP	34
4.4.	Giải thích toàn cục	35
4.5.	Hướng đóng góp của đặc trưng	36
4.6.	Giải thích một dự đoán cụ thể	37
5.	Kết quả thực nghiệm	39
5.1.	Thiết lập thực nghiệm	39
5.2.	Bảng tổng hợp	39
5.3.	Phân tích	40
6.	Xây dựng demo	41
6.1.	Cài đặt ứng dụng	41
6.2.	Cài đặt môi trường và khởi chạy	44

7.	Tài liệu tham khảo	46
8.	Phụ lục	48

Riêng với dữ liệu dạng bảng (tabular data), nhóm mô hình dựa trên cây quyết định (tree-based models) từ lâu vẫn là lựa chọn mặc định. Nhiều nghiên cứu so sánh trên quy mô lớn cho thấy các mô hình cây vượt trội so với các mô hình không thuộc họ cây trên phần lớn bộ dữ liệu bảng, cả về độ chính xác lẫn chi phí huấn luyện [5, 6]. Đặc điểm này đến từ bản chất của cây: chia không gian đặc trưng thành các vùng đơn giản bằng các luật “nếu-thì”, không giả định về dạng phân phối, xử lý tốt đặc trưng dị chất (heterogeneous features) và giữ được khả năng diễn giải.

1.2. Mục tiêu và phạm vi

Dự án này đặt trọng tâm vào hai công việc:

- (i) **Phân tích bài toán chẩn đoán bệnh tim** trên bộ dữ liệu Cleveland Heart Disease, bao gồm khám phá dữ liệu, tiền xử lý và xây dựng các biến thể dữ liệu khác nhau bằng feature engineering và feature selection.
- (ii) **Benchmark các mô hình machine learning, với trọng tâm là họ cây** trên cùng một giao thức thực nghiệm, nhằm phân tích ảnh hưởng của lựa chọn mô hình và cách xử lý đặc trưng đối với bài toán dữ liệu bảng có số mẫu nhỏ và nhãn nhị phân.

Các mô hình dự đoán được đưa vào benchmark gồm:

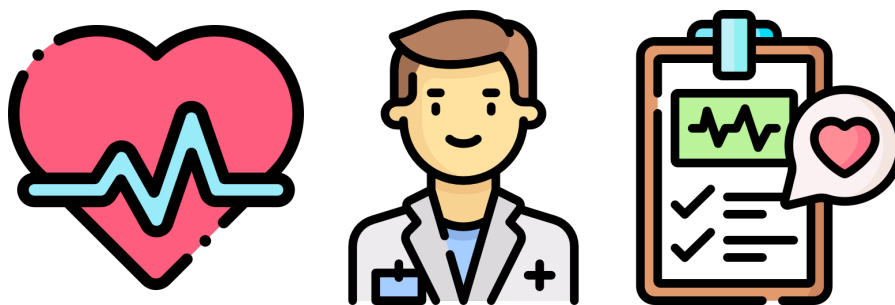
- **Naive Bayes** và **K-Nearest Neighbors (KNN)** đóng vai trò các baseline không thuộc họ cây để cung cấp mốc tham chiếu.
- **Decision Tree** là mô hình cây đơn lẻ, đóng vai trò baseline của họ tree-based models.
- **Random Forest** đại diện cho họ Bagging, kết hợp nhiều cây quyết định được ngẫu nhiên hóa nhằm giảm phương sai.
- **AdaBoost** và **Gradient Boosting** đại diện cho họ Boosting, trong đó các weak learners được xây dựng tuần tự để cải thiện lỗi của mô hình tổng hợp.
- **XGBoost** là một hiện thực tối ưu hóa của gradient boosting với regularization và nhiều cải tiến về hiệu quả tính toán.

Sau khi benchmark các mô hình dự đoán, chúng ta sẽ sử dụng **Tree SHapley Additive exPlanations (TreeSHAP)** [7, 8] để phân tích và giải thích dự đoán của mô hình cây được lựa chọn. TreeSHAP không phải một mô hình phân loại độc lập mà là phương pháp post-hoc explainability dành cho các mô hình dựa trên cây.

2. Bộ dữ liệu và tiền xử lý

2.1. Bộ dữ liệu Cleveland Heart Disease

“Biết người biết ta, trăm trận trăm thắng”. Trong machine learning, việc hiểu rõ dữ liệu là yếu tố quyết định để xây dựng mô hình hiệu quả. Dự án sử dụng bộ dữ liệu Cleveland Heart Disease, một trong bốn tập con thuộc bộ dữ liệu bệnh tim tổng hợp do UCI Machine Learning Repository cung cấp [9]. So với các tập con khác (Hungary, Switzerland, Long Beach VA), Cleveland được lựa chọn rộng rãi trong nghiên cứu nhờ có ít giá trị thiếu, nhãn phân loại rõ ràng và xuất phát từ nghiên cứu y học kinh điển của Detrano và cộng sự [10].



Cleveland Heart Disease Diagnosis

Hình 2: Bộ dữ liệu chẩn đoán bệnh tim Cleveland Heart Disease.

Bộ dữ liệu gồm 303 bệnh nhân với 13 đặc trưng đầu vào và một nhãn mục tiêu. Các đặc trưng bao gồm thông tin nhân khẩu học (tuổi, giới tính), các chỉ số lâm sàng (huyết áp, cholesterol, nhịp tim tối đa), kết quả điện tâm đồ, đặc điểm đoạn ST, số mạch máu chính quan sát được qua fluoroscopy và kết quả thallium stress test. Mỗi đặc trưng đã được mã hóa dạng số để thuận tiện cho việc huấn luyện mô hình. Bảng 1 minh họa một vài mẫu dữ liệu lấy từ bộ gốc.

Bảng 1: Một số mẫu dữ liệu từ bộ Cleveland Heart Disease.

age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
63	1	1	145	233	1	2	150	0	2.3	3	0.0	6.0	0
67	1	4	160	286	0	2	108	1	1.5	2	3.0	3.0	1
67	1	4	120	229	0	2	129	1	2.6	2	2.0	7.0	1
37	1	3	130	250	0	0	187	0	3.5	3	0.0	3.0	0
41	0	2	130	204	0	2	172	0	1.4	1	0.0	3.0	0

Bảng mô tả đặc trưng bộ dữ liệu Cleveland Heart Disease

Đặc trưng	Mô tả và giá trị mã hóa
age	Tuổi của bệnh nhân (năm).
sex	Giới tính. (1 = nam, 0 = nữ)
cp	Loại đau ngực. (1 = đau thắt ngực điển hình, 2 = không điển hình, 3 = đau không do tim, 4 = không triệu chứng)
trestbps	Huyết áp tâm thu lúc nghỉ (mmHg).
chol	Nồng độ cholesterol huyết thanh (mg/dL).
fbs	Đường huyết lúc đói > 120 mg/dL. (1 = đúng, 0 = sai)
restecg	Điện tâm đồ lúc nghỉ. (0 = bình thường, 1 = bất thường ST-T, 2 = phì đại thất trái)
thalach	Nhịp tim tối đa đạt được (lần/phút).
exang	Đau ngực khi gắng sức. (1 = có, 0 = không)
oldpeak	Mức độ trầm ST do gắng sức so với lúc nghỉ.
slope	Độ dốc đoạn ST. (1 = dốc lên, 2 = bằng phẳng, 3 = dốc xuống)
ca	Số mạch máu chính được nhuộm màu (0–3).
thal	Kết quả thallium stress test. (3 = bình thường, 6 = khuyết tật cố định, 7 = khuyết tật có thể hồi phục)
num	Nhãn mục tiêu. (0 = không bệnh, 1–4 = có bệnh)

Các giá trị rời rạc được mã hóa trong ngoặc để phục vụ huấn luyện mô hình.

Nhãn mục tiêu gốc nhận giá trị từ 0 đến 4, trong đó 0 là không mắc bệnh và 1–4 thể hiện các mức độ tổn thương mạch vành khác nhau. Trong toàn bộ dự án, nhãn được nhị phân hóa thành hai lớp: 0 (không bệnh) và 1 (có bệnh), tương ứng với mọi giá trị lớn hơn 0. Cách làm này thống nhất với đa số công trình sử dụng bộ Cleveland và giúp bài toán trở thành phân loại nhị phân.

2.2. Khám phá dữ liệu

Trước khi tiến xử lý, cần nạp các thư viện và đặt seed để kết quả có thể lặp lại ổn định giữa các lần chạy.

Nạp thư viện và đặt seed

```
1 import os
2 import random
3 import numpy as np
4 import pandas as pd
5 import seaborn as sns
6 import matplotlib.pyplot as plt
7
8 from pathlib import Path
9 from sklearn.pipeline import Pipeline
10 from sklearn.impute import SimpleImputer
11 from sklearn.compose import ColumnTransformer
12 from sklearn.tree import DecisionTreeClassifier
13 from sklearn.model_selection import train_test_split
14 from sklearn.base import BaseEstimator, TransformerMixin
15 from sklearn.feature_selection import mutual_info_classif
16 from sklearn.preprocessing import (
17     OneHotEncoder, StandardScaler, MinMaxScaler
18 )
19
20 SEED = 42
21 os.environ["PYTHONHASHSEED"] = str(SEED)
22 np.random.seed(SEED)
23 random.seed(SEED)
24 print(f"Seed: {SEED}")
```

Tiếp theo là tải tập dữ liệu Cleveland bằng `gdown`, đặt tên cột theo chuẩn, ép kiểu số cho các trường quan trọng và nhị phân hóa cột target.

Tải bộ dữ liệu Cleveland

```
1 !gdown 1U02zDTnnSG2Xf_YINxs-FqfR7UbdjeNI
```

Đọc và cấu trúc hóa dữ liệu

```
1 DATA_PATH = "[Data]-cleveland.csv"
2 COLUMNS = ["age", "sex", "cp", "trestbps", "chol", "fbs", "restecg",
3             "thalach", "exang", "oldpeak", "slope", "ca", "thal", "target"]
4
5 numeric_cols = ["age", "trestbps", "chol", "thalach", "oldpeak"]
6 categorical_cols = ["sex", "cp", "fbs", "restecg", "exang", "slope", "ca", "thal"]
7
8 TARGET = "target"
9 K_FEATURES = 10
10
11 raw = pd.read_csv(DATA_PATH, header=None)
12 raw.columns = COLUMNS
13
14 for c in ["age", "trestbps", "chol", "thalach", "oldpeak", "ca", "thal"]:
15     raw[c] = pd.to_numeric(raw[c], errors="coerce")
16
17 raw[TARGET] = (raw[TARGET] > 0).astype(int)
```

```

18
19 print("Shape:", raw.shape)
20 display(raw.head())
21 display(raw.isna().sum())

```

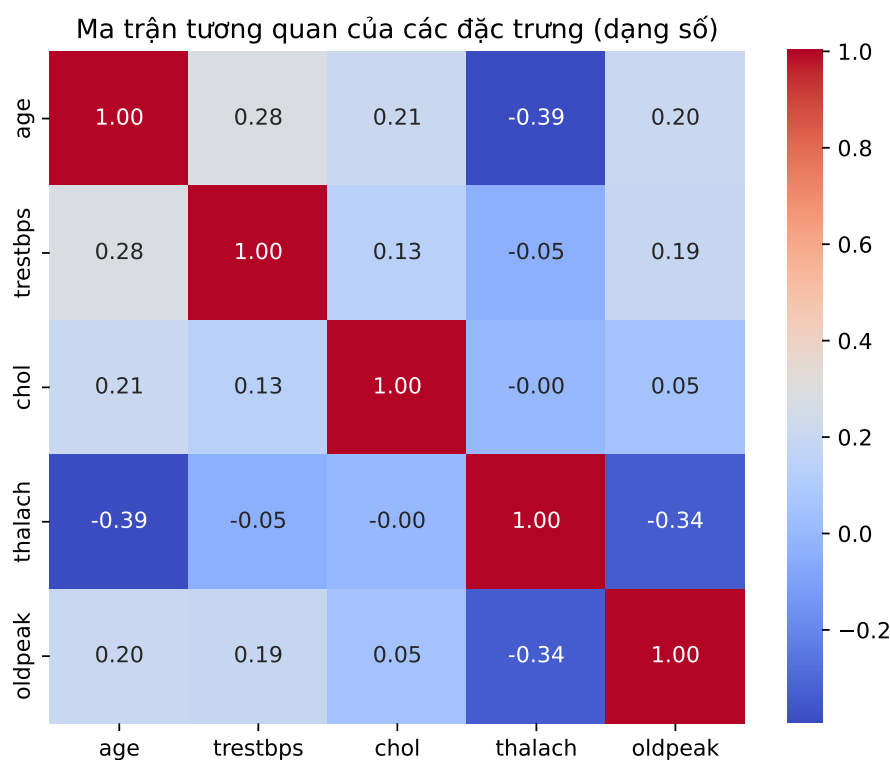
Bước khám phá dữ liệu tập trung vào ma trận tương quan giữa các đặc trưng dạng số. Mục đích là phát hiện các cặp đặc trưng có tương quan cao (dấu hiệu của thông tin dư thừa) và gợi ý các đặc trưng tiềm năng để tạo mới ở bước sau.

Ma trận tương quan

```

1 plt.figure(figsize=(6, 5))
2 corr_matrix = raw[numeric_cols].corr()
3 sns.heatmap(corr_matrix, annot=True, cmap="coolwarm", fmt=".2f")
4 plt.title("Ma trận tương quan của các đặc trưng (dạng số)")
5 plt.tight_layout()
6 plt.show()

```

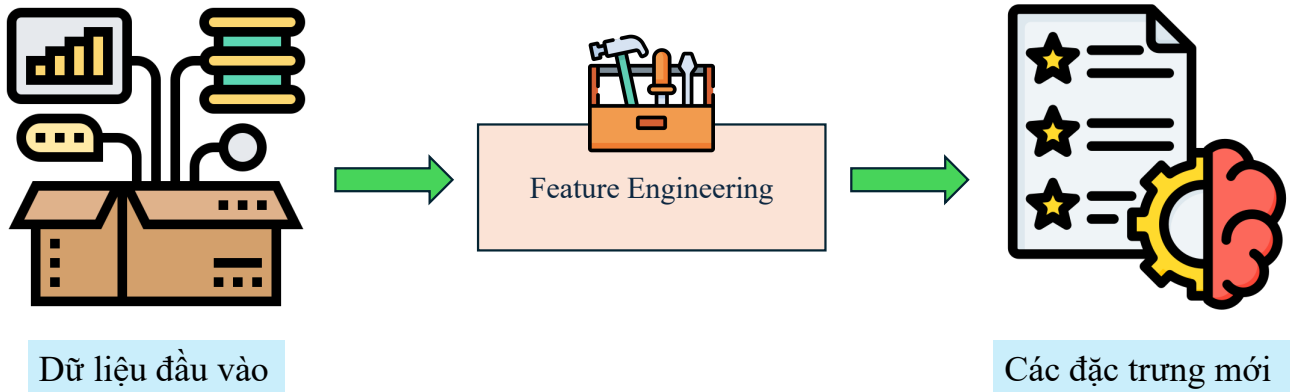


Hình 3: Ma trận tương quan giữa các đặc trưng dạng số.

Có thể thấy các đặc trưng dạng số tương quan với nhau ở mức thấp, không có cặp nào vượt ngưỡng cần loại bỏ. Đáng chú ý là **age** có tương quan âm rõ nhất với **thalach**, phản ánh quy luật sinh lý rằng nhịp tim tối đa giảm dần theo tuổi. Quan sát này là cơ sở trực tiếp cho các đặc trưng tỷ lệ theo tuổi được tạo ở phần 2.3..

2.3. Xây dựng bốn biến thể dữ liệu

Feature engineering (FE) là kỹ thuật biến đổi và bổ sung đặc trưng từ dữ liệu gốc để mô hình có khả năng học tốt hơn. Việc này giúp làm nổi bật các tín hiệu hữu ích, giảm nhiễu, đồng thời tạo ra các đặc trưng mới có giá trị phân tách cao hơn cho quá trình huấn luyện.



Hình 4: Minh họa kỹ thuật feature engineering khi áp dụng vào một bộ dữ liệu nhằm tạo thêm các đặc trưng mới tốt hơn.

Để đánh giá tác động của tiền xử lý lên các mô hình cây, dự án xây dựng bốn biến thể dữ liệu từ cùng một phép chia train/val/test:

- (i) **Original** là bộ gốc sau khi imputation và scaling, giữ nguyên 13 đặc trưng.
- (ii) **FE** là bộ gốc cộng thêm các đặc trưng sinh mới, mã hóa one-hot cho biến phân loại, sau đó chọn Top- K đặc trưng theo mutual information (MI).
- (iii) **Original + DT** là bộ Original được rút gọn còn $K = 10$ đặc trưng quan trọng nhất theo feature importance của Decision Tree.
- (iv) **FE + DT** là bộ FE được rút gọn theo cùng cách.

Điểm cần lưu ý là hai cơ chế chọn đặc trưng ở đây khác nhau về bản chất. Mutual information đo mức độ phụ thuộc thống kê giữa từng đặc trưng và nhãn một cách độc lập, trong khi feature importance của Decision Tree đo mức giảm impurity mà đặc trưng đó mang lại trong ngữ cảnh của các đặc trưng khác. Việc so sánh hai cơ chế này trên cùng bài toán là một phần nội dung thực nghiệm.

2.3.1. Chia tập và tiền xử lý bộ Original

Dữ liệu được chia thành ba tập train, validation và test theo tỉ lệ 80:10:10, có phân tầng (stratify) theo nhãn để giữ nguyên tỉ lệ hai lớp trong mọi tập.

Chia tập dữ liệu

```
1 raw_feature_cols = [c for c in raw.columns if c != TARGET]
2 X_all = raw[raw_feature_cols]
3 y_all = raw[TARGET]
4
```

```

5 X_train, X_temp, y_train, y_temp = train_test_split(
6     X_all, y_all, test_size=0.2, stratify=y_all, random_state=SEED
7 )
8 X_val, X_test, y_val, y_test = train_test_split(
9     X_temp, y_temp, test_size=0.5, stratify=y_temp, random_state=SEED
10 )
11 print(X_train.shape, X_val.shape, X_test.shape)

```

Luồng tiền xử lý được tách theo kiểu dữ liệu:

- Cột dạng số: thay giá trị thiếu bằng trung vị (median), sau đó chuẩn hóa về phân phối có trung bình 0 và độ lệch chuẩn 1 (StandardScaler).
- Cột dạng phân loại: thay giá trị thiếu bằng giá trị xuất hiện nhiều nhất (mode), sau đó chuẩn hóa theo Min-Max.

Toàn bộ tham số của các bước này chỉ được học từ tập train (`fit_transform`), rồi áp dụng lại cho val và test (`transform`). Đây là điểm mấu chốt để tránh data leakage.

Xây dựng bộ Original

```

1 num_proc = Pipeline([
2     ("imputer", SimpleImputer(strategy="median")),
3     ("scaler", StandardScaler())
4 ])
5 cat_proc = Pipeline([
6     ("imputer", SimpleImputer(strategy="most_frequent")),
7     ("scaler", MinMaxScaler())
8 ])
9
10 preprocess = ColumnTransformer([
11     ("num", num_proc, numeric_cols),
12     ("cat", cat_proc, categorical_cols),
13 ], verbose_feature_names_out=False).set_output(transform="pandas")
14
15 raw_pipeline = Pipeline([("preprocess", preprocess)])
16
17 X_raw_train = raw_pipeline.fit_transform(X_train, y_train)
18 X_raw_val = raw_pipeline.transform(X_val)
19 X_raw_test = raw_pipeline.transform(X_test)
20
21 out_dir = Path("splits")
22 out_dir.mkdir(parents=True, exist_ok=True)
23
24 for name, (X_, y_) in {
25     "raw_train": (X_raw_train, y_train),
26     "raw_val": (X_raw_val, y_val),
27     "raw_test": (X_raw_test, y_test),
28 }.items():
29     pd.concat(
30         [X_.reset_index(drop=True), y_.reset_index(drop=True).rename(TARGET)],
31         axis=1
32     ).to_csv(out_dir / f"{name}.csv", index=False)
33

```

```
34 display(X_raw_train.head())
```

2.3.2. Bộ FE với đặc trưng sinh mới

Các đặc trưng mới trong phần này được xây dựng với mục đích khám phá (exploratory feature engineering), nhằm biểu diễn một số tương tác đơn giản giữa tuổi và các chỉ số lâm sàng. Cụ thể, các tỷ lệ như `chol_per_age`, `bps_per_age` và `hr_ratio` được sử dụng như các biểu diễn bổ sung để kiểm tra liệu việc chuẩn hóa tương đối theo tuổi có giúp mô hình phân tách hai lớp tốt hơn hay không.

Sinh đặc trưng mới và mã hóa one-hot

```
1 class AddNewFeaturesTransformer(BaseEstimator, TransformerMixin):
2     def __init__(self, n_age_bins=5):
3         self.n_age_bins = n_age_bins
4
5     def fit(self, X, y=None):
6         X = X.copy()
7
8         age_values = pd.to_numeric(
9             X["age"], errors="coerce"
10        ).dropna()
11
12        _, edges = pd.qcut(
13            age_values,
14            q=self.n_age_bins,
15            retbins=True,
16            duplicates="drop"
17        )
18
19        # Bảo đảm mọi giá trị val/test đều có thể được ánh xạ.
20        edges[0] = -np.inf
21        edges[-1] = np.inf
22        self.age_bin_edges_ = edges
23
24        return self
25
26    def transform(self, X):
27        df = X.copy()
28
29        if {"chol", "age"} <= set(df.columns):
30            df["chol_per_age"] = df["chol"] / df["age"]
31
32        if {"trestbps", "age"} <= set(df.columns):
33            df["bps_per_age"] = df["trestbps"] / df["age"]
34
35        if {"thalach", "age"} <= set(df.columns):
36            df["hr_ratio"] = df["thalach"] / df["age"]
37
38        if "age" in df.columns:
39            df["age_bin"] = pd.cut(
40                df["age"],
41                bins=self.age_bin_edges_,
42                labels=False,
```

```

43         include_lowest=True
44     ).astype("category")
45
46     return df
47
48     def get_feature_names_out(self, input_features=None):
49         return list(input_features) + [
50             "chol_per_age",
51             "bps_per_age",
52             "hr_ratio",
53             "age_bin"
54         ]
55
56
57 gen_num = ["chol_per_age", "bps_per_age", "hr_ratio"]
58 gen_cat = ["age_bin"]
59 all_nums = numeric_cols + gen_num
60 all_cats = categorical_cols + gen_cat
61
62 fe_num_proc = Pipeline([
63     ("imp", SimpleImputer(strategy="median")),
64     ("sc", StandardScaler())
65 ])
66 fe_cat_proc = Pipeline([
67     ("imp", SimpleImputer(strategy="most_frequent")),
68     ("ohe", OneHotEncoder(handle_unknown="ignore", sparse_output=False))
69 ])
70
71 fe_pre = Pipeline([
72     ("add", AddNewFeaturesTransformer(n_age_bins=5)),
73     ("pre", ColumnTransformer([
74         ("num", fe_num_proc, all_nums),
75         ("cat", fe_cat_proc, all_cats),
76     ], verbose_feature_names_out=False).set_output(transform="pandas")),
77 ]).set_output(transform="pandas")
78
79 Xt_tr = fe_pre.fit_transform(X_train, y_train)
80 Xt_va = fe_pre.transform(X_val)
81 Xt_te = fe_pre.transform(X_test)
82
83 nz_cols = Xt_tr.columns[Xt_tr.nunique(dropna=False) > 1]
84 Xt_tr, Xt_va, Xt_te = Xt_tr[nz_cols], Xt_va[nz_cols], Xt_te[nz_cols]
85 print("Số đặc trưng sau FE:", Xt_tr.shape[1])

```

Sau khi mã hóa one-hot, số chiều tăng lên đáng kể. Mutual information được dùng để xếp hạng và giữ lại Top- K đặc trưng, với K bằng số đặc trưng của bộ gốc để hai bộ dữ liệu có cùng số chiều và việc so sánh trở nên công bằng.

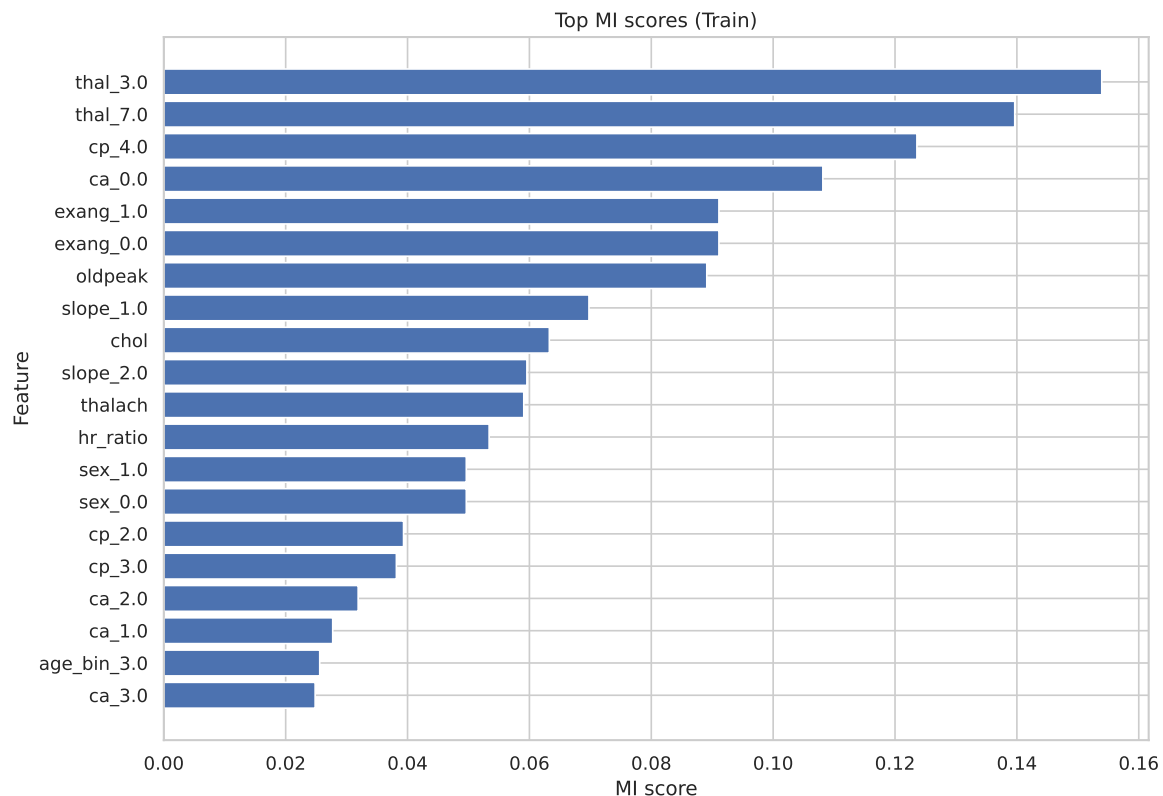
Chọn đặc trưng theo Mutual Information

```

1 ohe = fe_pre.named_steps["pre"].named_transformers_["cat"].named_steps["ohe"]
2 cat_names = list(ohe.get_feature_names_out(all_cats))
3 is_discrete = np.array([c in cat_names for c in Xt_tr.columns], dtype=bool)
4

```

```
5 mi = mutual_info_classif(
6     Xt_tr.values, y_train.values,
7     discrete_features=is_discrete, random_state=SEED
8 )
9 mi_series = pd.Series(mi, index=Xt_tr.columns).sort_values(ascending=False)
10
11 K = len(raw_feature_cols)
12 topk_cols = list(mi_series.head(K).index)
13
14 for name, (X_, y_) in {
15     "fe_train": (Xt_tr[topk_cols], y_train),
16     "fe_val": (Xt_va[topk_cols], y_val),
17     "fe_test": (Xt_te[topk_cols], y_test),
18 }.items():
19     pd.concat(
20         [X_.reset_index(drop=True), y_.reset_index(drop=True).rename(TARGET)],
21         axis=1
22     ).to_csv(out_dir / f"{name}.csv", index=False)
23
24 N = min(20, len(mi_series))
25 topN = mi_series.head(N).iloc[::-1]
26 plt.figure(figsize=(10, max(6, 0.35 * N)))
27 plt.barh(topN.index, topN.values)
28 plt.title("Top MI scores (Train)")
29 plt.xlabel("MI score")
30 plt.ylabel("Feature")
31 plt.tight_layout()
32 plt.show()
```



Hình 5: Xếp hạng đặc trưng theo mutual information trên tập train.

2.3.3. Chọn đặc trưng bằng Decision Tree

Cuối cùng, một Decision Tree được huấn luyện trên mỗi bộ dữ liệu để lấy feature importance, từ đó rút gọn còn $K = 10$ đặc trưng quan trọng nhất. Hàm dưới đây được viết chung để dùng lại cho cả hai bộ Original và FE.

Chọn Top-K đặc trưng bằng Decision Tree

```

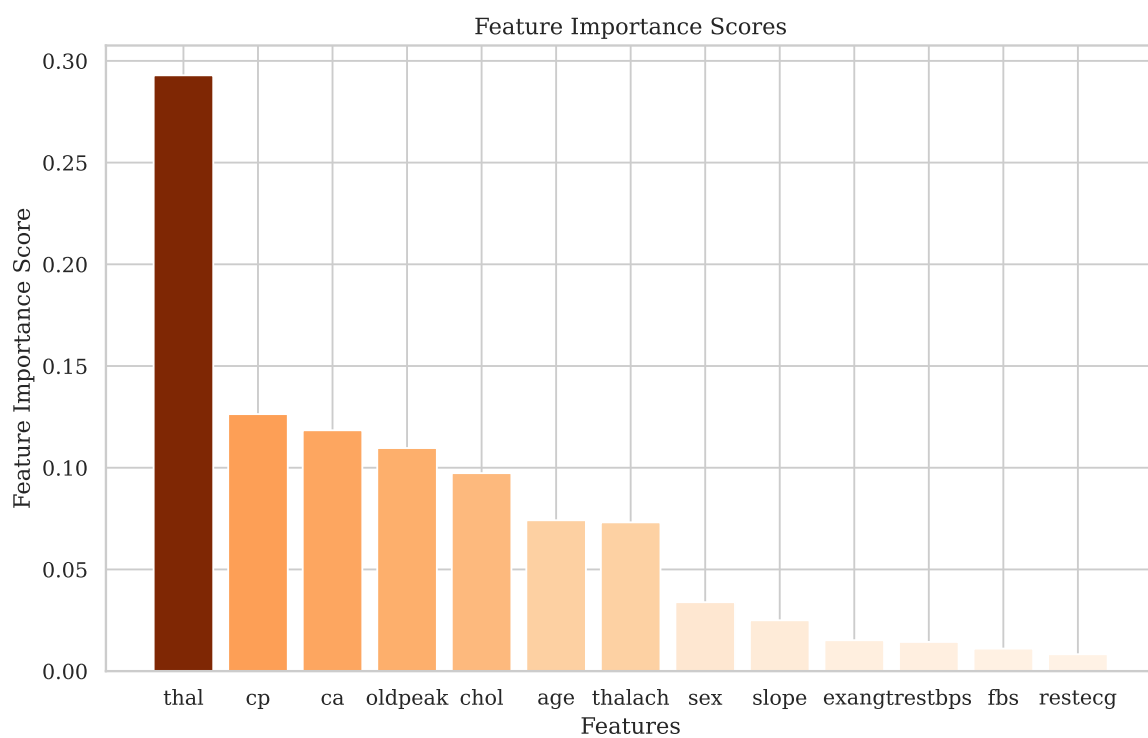
1 def select_by_dt(X_tr, X_va, X_te, y_tr, k=K_FEATURES, prefix="dt"):
2     dt = DecisionTreeClassifier(random_state=SEED)
3     dt.fit(X_tr, y_tr)
4
5     importance = pd.Series(
6         dt.feature_importances_, index=X_tr.columns
7     ).sort_values(ascending=False)
8     selected = importance.head(k).index.tolist()
9     print(f"Top {k} đặc trưng ({prefix}): {selected}")
10
11     subsets = {
12         f"{prefix}_train": (X_tr[selected], y_tr),
13         f"{prefix}_val": (X_va[selected], y_val),
14         f"{prefix}_test": (X_te[selected], y_test),
15     }
16     for name, (X_, y_) in subsets.items():
17         pd.concat(
18             [X_.reset_index(drop=True),
19              y_.reset_index(drop=True).rename(TARGET)],
20             axis=1

```

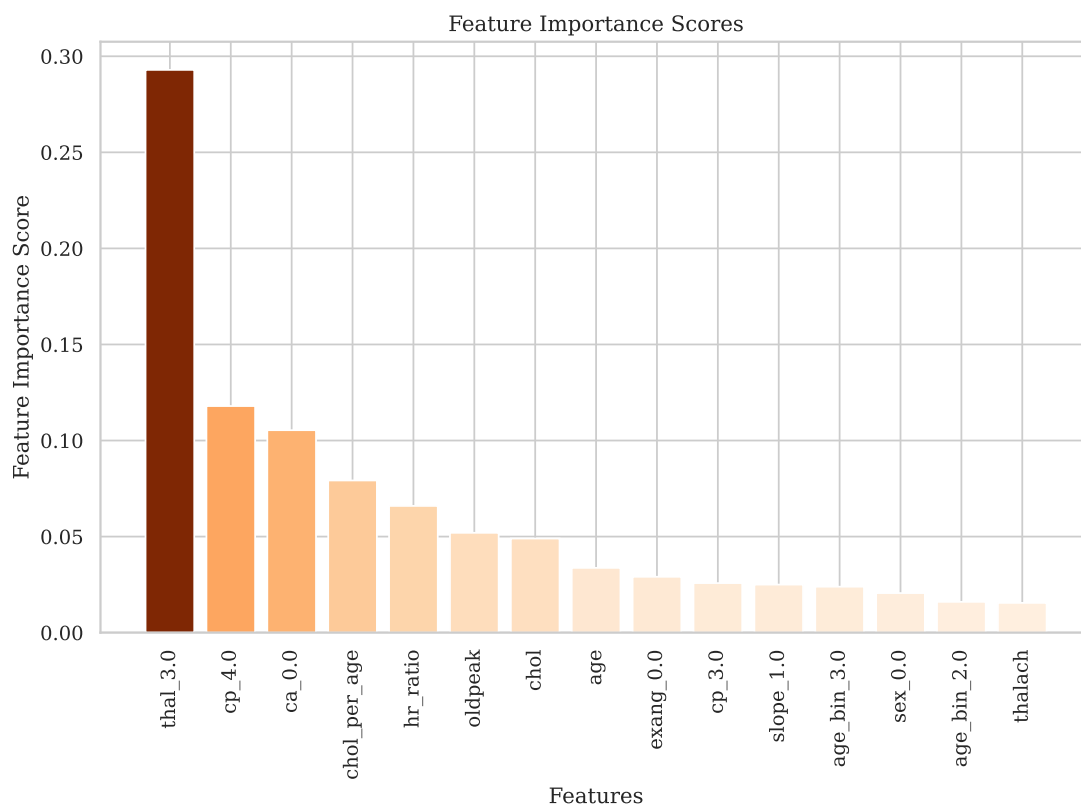
```

21         ).to_csv(out_dir / f"{name}.csv", index=False)
22
23     return importance, selected
24
25
26 imp_raw, sel_raw = select_by_dt(
27     X_raw_train, X_raw_val, X_raw_test, y_train, prefix="dt"
28 )
29 imp_fe, sel_fe = select_by_dt(
30     Xt_tr[topk_cols], Xt_va[topk_cols], Xt_te[topk_cols],
31     y_train, prefix="fe_dt"
32 )

```



Hình 6: Mức độ ảnh hưởng của các đặc trưng theo Decision Tree trên bộ Original.



Hình 7: Mức độ ảnh hưởng của các đặc trưng theo Decision Tree trên bộ FE.

So sánh Hình 5 và Hình 6 cho thấy sự khác biệt về nguyên lý đã nêu ở trên. Mutual information xếp hạng từng đặc trưng one-hot riêng lẻ (ví dụ **thal_3.0**) theo mức phụ thuộc thống kê với nhãn, trong khi Decision Tree đánh giá đặc trưng theo mức giảm impurity tại các điểm chia, do đó phản ánh cả tương tác giữa các đặc trưng.

Sau khi hoàn tất, nén thư mục **splits** thành một tệp ZIP duy nhất chứa 12 tệp CSV, tương ứng với 4 biến thể dữ liệu và 3 tập con.

Nén các tập dữ liệu

```
1 !zip -r dataset.zip splits
```


3. Benchmark các mô hình Machine Learning

Phần này lần lượt trình bày các mô hình được đưa vào benchmark. Toàn bộ thực nghiệm dùng chung một khung đánh giá, được xây dựng ngay ở đầu phần và dùng lại cho mọi mô hình.

3.1. Khung thực nghiệm chung

Lưu ý: Các khối code được đánh dấu “*” nghĩa là được sử dụng lại nhiều lần mà không cần sửa đổi giữa các mô hình khác nhau trong dự án này.

* Nạp thư viện và đặt seed

```
1 import os
2 import random
3 import warnings
4 import numpy as np
5 import pandas as pd
6 import matplotlib.pyplot as plt
7
8 from xgboost import XGBClassifier
9 from sklearn.naive_bayes import GaussianNB
10 from sklearn.tree import DecisionTreeClassifier
11 from sklearn.neighbors import KNeighborsClassifier
12 from sklearn.ensemble import (
13     RandomForestClassifier, AdaBoostClassifier, GradientBoostingClassifier
14 )
15 from sklearn.metrics import accuracy_score, f1_score, classification_report
16 from sklearn.model_selection import StratifiedKFold, cross_val_score
17
18 warnings.filterwarnings("ignore")
19
20 SEED = 42
21 os.environ["PYTHONHASHSEED"] = str(SEED)
22 np.random.seed(SEED)
23 random.seed(SEED)
24 print(f"Seed: {SEED}")
```

* Tải và đọc các bộ dữ liệu

```
1 !gdown --folder 1cMoqIDEgGYDVzv8B7cKp3csxujQ40Fp7
2
3
4 def read_csv(file_path):
5     df = pd.read_csv(file_path)
6     X = df.drop(TARGET, axis=1)
7     y = df[TARGET]
8     print(f"{file_path}: X={X.shape}, y={y.shape}")
9     return X, y
10
11
12 DATASETS = {}
13 for tag, prefix in [
14     ("Original", "raw"),
15     ("FE", "fe"),
```

```

16     ("Original+DT", "dt"),
17     ("FE+DT", "fe_dt"),
18 ]:
19     DATASETS[tag] = {
20         split: read_csv(f"splits/{prefix}_{split}.csv")
21         for split in ["train", "val", "test"]
22     }

```

Việc gom bốn biến thể dữ liệu vào một dictionary cho phép viết một vòng lặp duy nhất thay vì lặp lại bốn lần cho mỗi mô hình. Hai hàm dưới đây là khung chung: một hàm dò siêu tham số bằng Stratified K-Fold Cross-Validation, một hàm chạy toàn bộ quy trình train, validate và test trên cả bốn bộ dữ liệu.

* Hàm dò siêu tham số bằng Stratified K-Fold CV

```

1 def tune_by_cv(make_model, param_values, X_train, y_train,
2                 param_name="n_estimators", cv_splits=5, plot_title=""):
3     cv = StratifiedKFold(n_splits=cv_splits, shuffle=True, random_state=SEED)
4     scores = []
5
6     for v in param_values:
7         model = make_model(v)
8         cv_score = cross_val_score(
9             model, X_train, y_train, cv=cv, scoring="accuracy", n_jobs=-1
10        )
11         scores.append(cv_score.mean())
12
13     plt.figure(figsize=(10, 6))
14     plt.plot(list(param_values), scores, "bo-")
15     plt.title(f"{plot_title} (CV={cv_splits}-fold)")
16     plt.xlabel(param_name)
17     plt.ylabel("Cross-Validation Accuracy")
18     plt.grid(True)
19     plt.show()
20
21     best_v = list(param_values)[int(np.argmax(scores))]
22     print(f"{param_name} tối ưu (CV): {best_v}")
23
24     best_model = make_model(best_v)
25     best_model.fit(X_train, y_train)
26     return best_model, best_v, max(scores)

```

* Hàm chạy benchmark trên bốn bộ dữ liệu

```

1 def run_benchmark(model_name, make_model, param_values,
2                   param_name="n_estimators", cv_splits=5):
3     results = {}
4
5     for tag, data in DATASETS.items():
6         print(f"\n===== {model_name} on {tag} =====")
7         X_tr, y_tr = data["train"]
8         X_va, y_va = data["val"]

```

```

9     X_te, y_te = data["test"]
10
11     model, best_v, _ = tune_by_cv(
12         make_model, param_values, X_tr, y_tr,
13         param_name=param_name, cv_splits=cv_splits,
14         plot_title=f"Chọn {param_name} tối ưu cho {model_name}"
15     )
16
17     val_pred = model.predict(X_va)
18     test_pred = model.predict(X_te)
19
20     results[tag] = {
21         "model": model,
22         "val_acc": accuracy_score(y_va, val_pred),
23         "test_acc": accuracy_score(y_te, test_pred),
24         "val_f1": f1_score(y_va, val_pred),
25         "test_f1": f1_score(y_te, test_pred),
26         "best_param": best_v,
27     }
28     print(f"Val acc: {results[tag]['val_acc']:.4f} | "
29           f"Test acc: {results[tag]['test_acc']:.4f}")
30     print(classification_report(y_te, test_pred))
31
32     return results

```

* Hàm vẽ biểu đồ so sánh

```

1 plt.rcParams["font.family"] = "DejaVu Serif"
2
3
4 def plot_results(results, model_name):
5     labels = list(results.keys())
6     val_accs = [results[k]["val_acc"] for k in labels]
7     test_accs = [results[k]["test_acc"] for k in labels]
8
9     x = np.arange(len(labels))
10    width = 0.3
11    fig, ax = plt.subplots(figsize=(6, 5))
12
13    rects1 = ax.bar(x - width/2, val_accs, width, label="Validation Accuracy",
14                   color="tab:blue", edgecolor="black", linewidth=1.2)
15    rects2 = ax.bar(x + width/2, test_accs, width, label="Test Accuracy",
16                   color="tab:red", edgecolor="black", linewidth=1.2)
17
18    ax.set_ylim(0.5, 1.05)
19    ax.set_ylabel("Accuracy")
20    ax.set_title(model_name, fontsize=16)
21    ax.set_xticks(x)
22    ax.set_xticklabels(labels, rotation=15)
23    ax.legend(ncol=2, loc="upper center")
24
25    for rects in (rects1, rects2):
26        for rect in rects:
27            h = rect.get_height()
28            ax.annotate(f"{h:.2f}",

```

```

29         xy=(rect.get_x() + rect.get_width()/2, h),
30         ha="center", va="bottom")
31
32     fig.tight_layout()
33     plt.show()

```

3.2. Baseline không thuộc họ cây: Naive Bayes và K-Nearest Neighbors

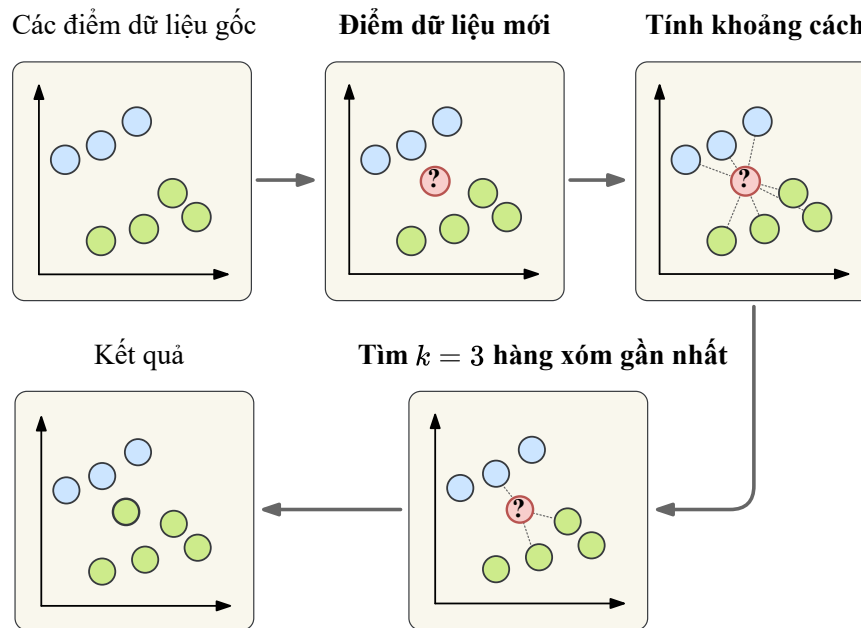
Trước khi đi vào họ cây, hai mô hình không thuộc họ cây được huấn luyện để làm mốc tham chiếu.

Naive Bayes là nhóm mô hình phân loại dựa trên định lý Bayes, tính xác suất hậu nghiệm của mỗi lớp và gán mẫu vào lớp có xác suất cao nhất. Mô hình áp dụng giả định “ngây thơ” (naive) rằng các đặc trưng đầu vào độc lập có điều kiện khi đã biết nhãn lớp. Nói cách khác, một khi biết bệnh nhân thuộc lớp “mắc bệnh tim” hay “không mắc bệnh tim”, giá trị của cholesterol sẽ không ảnh hưởng đến giá trị của huyết áp trong việc ước lượng xác suất thuộc lớp. Giả định này rõ ràng không đúng với dữ liệu y tế, nhưng mô hình vẫn cho kết quả chấp nhận được khi số mẫu hạn chế.

$$\begin{array}{c}
 \text{Khả năng xảy ra} \quad \text{Xác suất tiên nghiệm} \\
 P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)} \\
 \text{Xác suất hậu nghiệm} \qquad \qquad \text{Bằng chứng}
 \end{array}$$

Hình 8: Công thức Naive Bayes và các thành phần liên quan.

K-Nearest Neighbors (KNN) là mô hình học có giám sát và phi tham số (non-parametric). Thuật toán không học một hàm mục tiêu cụ thể mà lưu trữ toàn bộ tập huấn luyện để sử dụng khi dự đoán. Khi phân loại một mẫu mới, KNN xác định k điểm gần nhất trong tập huấn luyện dựa trên một thước đo khoảng cách, sau đó gán nhãn theo đa số phiếu. Giá trị k quá nhỏ khiến mô hình nhạy cảm với nhiễu, còn k quá lớn làm mờ ranh giới giữa các lớp.



Hình 9: Tổng quan các bước trong mô hình KNN.

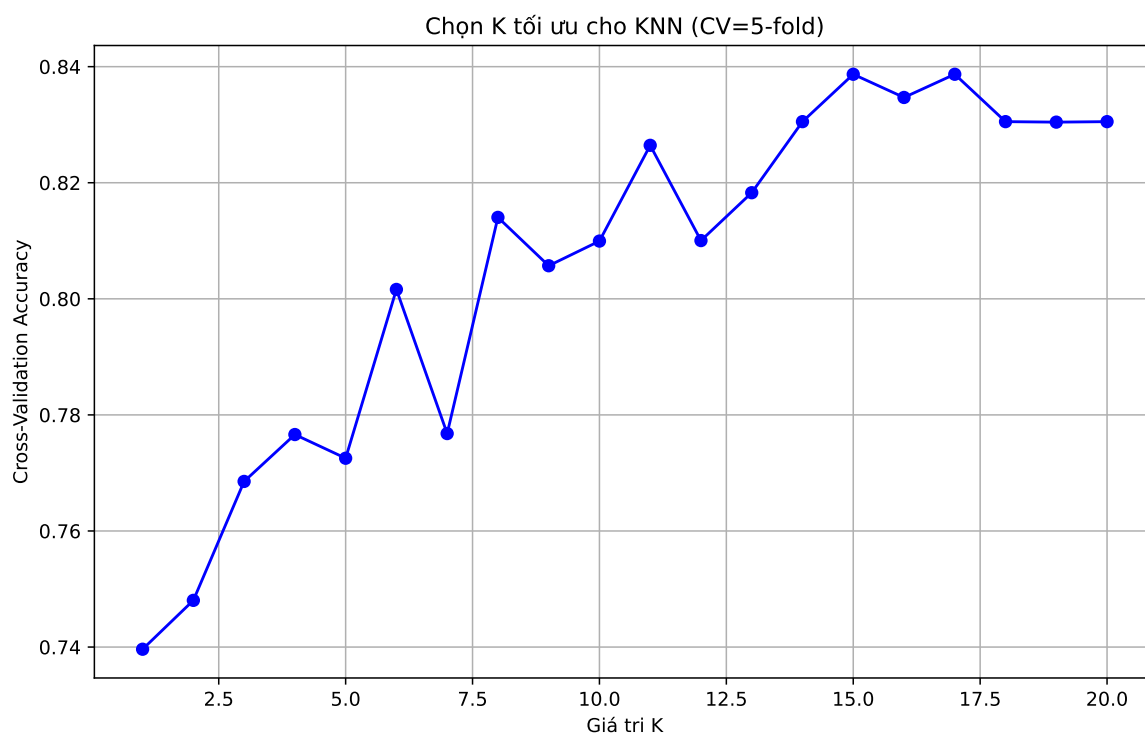
Cả hai mô hình được chạy qua khung benchmark chung. Naive Bayes không có siêu tham số cần dò nên chỉ truyền vào một giá trị duy nhất.

Baseline không thuộc họ cây

```

1 # Naive Bayes
2 nb_results = run_benchmark(
3     "GaussianNB",
4     make_model=lambda _: GaussianNB(),
5     param_values=[None],
6     param_name="(no hyperparameter)"
7 )
8 plot_results(nb_results, "GaussianNB")
9
10 # K-Nearest Neighbors
11 knn_results = run_benchmark(
12     "KNN",
13     make_model=lambda k: KNeighborsClassifier(n_neighbors=k),
14     param_values=range(1, 21),
15     param_name="k"
16 )
17 plot_results(knn_results, "KNN")

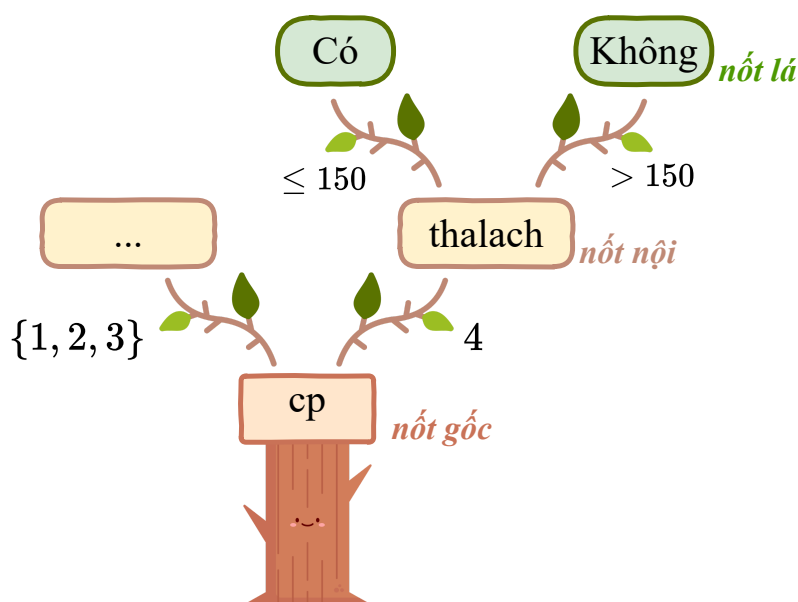
```



Hình 10: Biểu đồ xác định k tối ưu cho KNN theo Stratified 5-fold Cross-Validation.

3.3. Decision Tree

3.3.1. Giới thiệu



Hình 11: Minh họa mô hình cây quyết định bằng các thuộc tính của bài toán chẩn đoán bệnh tim.

Decision Tree là mô hình học có giám sát sử dụng cấu trúc dạng cây để đưa ra quyết định phân loại. Mô hình chia nhỏ không gian đặc trưng thành các vùng bằng cách áp dụng các quy

tắc “nếu-thì” tại mỗi nút. Mỗi **internal node** biểu diễn một phép kiểm tra trên một đặc trưng, mỗi nhánh thể hiện kết quả của phép kiểm tra đó, và mỗi **leaf node** đại diện cho nhãn dự đoán. Việc xây dựng cây dựa trên việc chọn đặc trưng và ngưỡng phân chia tối ưu tại mỗi bước, nhằm tối đa hóa độ thuần khiết của các nút con. Hai tiêu chí phổ biến là **Gini impurity** và **entropy**. Với một nút chứa tập mẫu S và p_i là tỉ lệ mẫu thuộc lớp i :

$$\text{Gini}(S) = 1 - \sum_{i=1}^C p_i^2, \quad H(S) = - \sum_{i=1}^C p_i \log_2 p_i$$

Information gain của một phép chia là mức giảm entropy sau khi chia:

$$IG(S, A) = H(S) - \sum_{v \in \text{values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

Nhờ tính trực quan và khả năng diễn giải cao, Decision Tree là mô hình nền tảng của toàn bộ họ cây được khảo sát trong dự án này. Nhược điểm chính là phương sai cao: một cây đơn lẻ rất nhạy với thay đổi nhỏ trong dữ liệu huấn luyện và dễ overfitting nếu không giới hạn độ sâu. Các phương pháp ở những phần tiếp theo đều nhằm khắc phục nhược điểm này theo những cách khác nhau.

3.3.2. Triển khai

Siêu tham số cần dò là độ sâu tối đa của cây (`max_depth`).

Decision Tree

```
1 dt_results = run_benchmark(
2     "Decision Tree",
3     make_model=lambda d: DecisionTreeClassifier(
4         max_depth=d, random_state=SEED
5     ),
6     param_values=range(2, 11),
7     param_name="max_depth"
8 )
9 plot_results(dt_results, "Decision Tree")
```

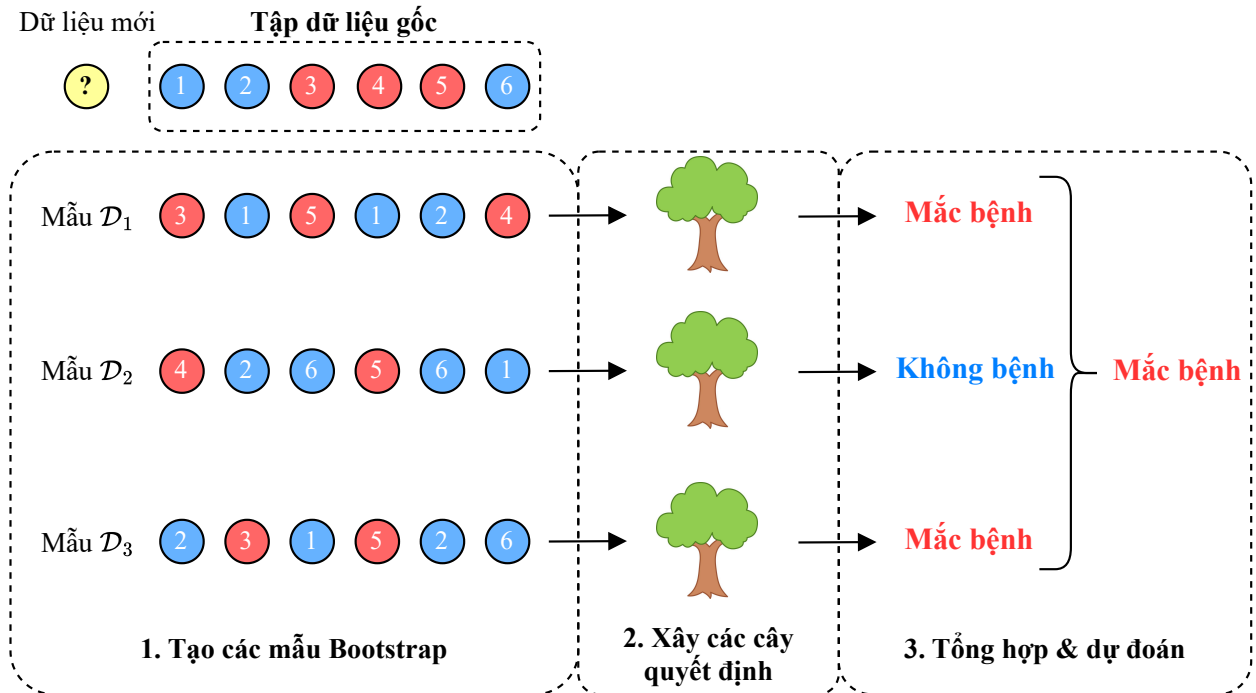
Ngoài độ chính xác, Decision Tree còn cho phép truy xuất mức độ đóng góp của từng đặc trưng, giúp đối chiếu kết quả mô hình với kiến thức y khoa.

Hiển thị feature importance

```
1 def display_feature_importance(model, X_train, top=10):
2     fi = pd.DataFrame({
3         "feature": X_train.columns,
4         "importance": model.feature_importances_
5     }).sort_values("importance", ascending=False)
6     display(fi.head(top))
7     return fi
```

3.4. Random Forest

3.4.1. Giới thiệu



Hình 12: Minh họa cách hoạt động của Random Forest.

Random Forest [11] là thuật toán thuộc nhóm ensemble, hoạt động bằng cách xây dựng một “rừng” gồm nhiều Decision Tree trong quá trình huấn luyện. Mục tiêu chính là cải thiện độ chính xác và khắc phục nhược điểm lớn của một cây đơn lẻ đã nêu ở phần 3.3.: xu hướng overfitting và phương sai cao.

Để đảm bảo các cây trong rừng khác biệt và không tương quan cao với nhau, Random Forest áp dụng hai kỹ thuật ngẫu nhiên hóa cốt lõi:

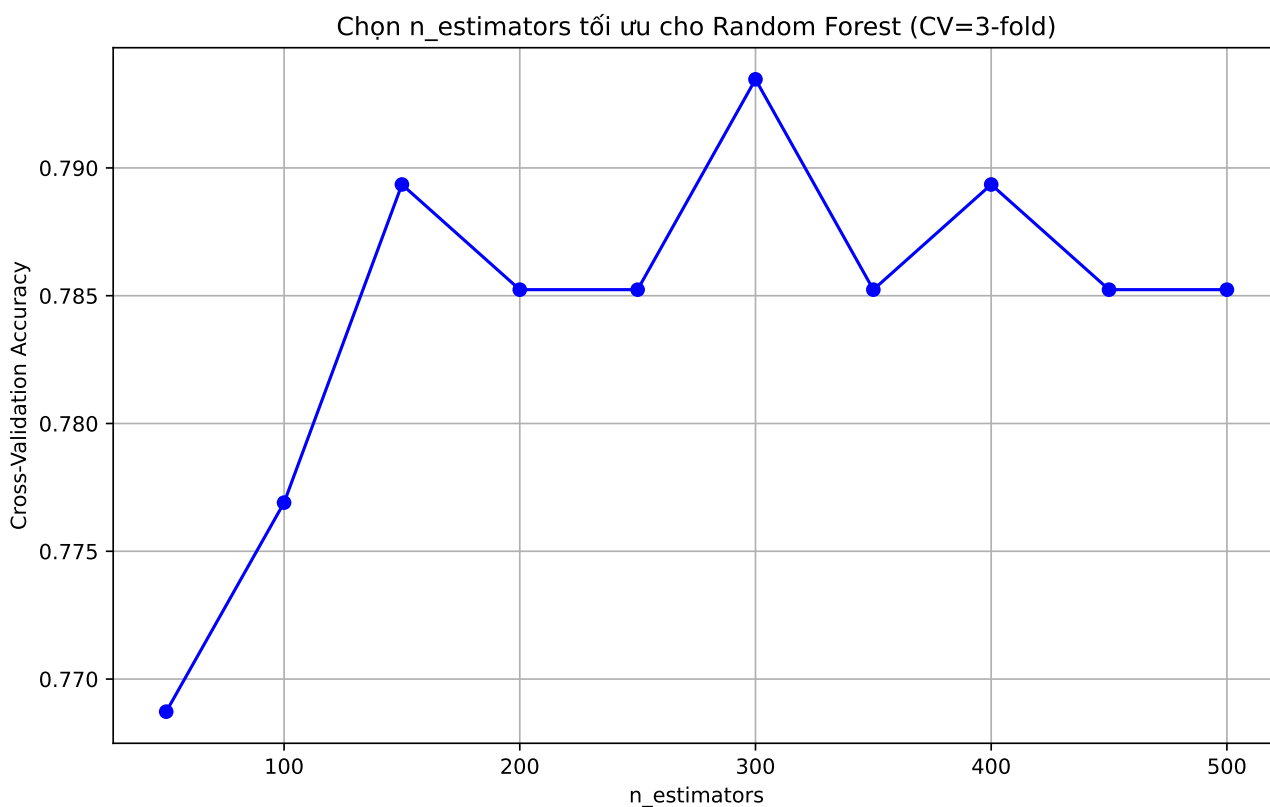
- Bagging (bootstrap aggregating)**: mỗi cây được huấn luyện trên một tập dữ liệu con khác nhau, lấy ngẫu nhiên có hoàn lại từ tập huấn luyện ban đầu.
- Feature randomness**: tại mỗi bước phân nhánh, thay vì xem xét toàn bộ các đặc trưng, thuật toán chỉ chọn một tập con ngẫu nhiên để tìm điểm chia tối ưu.

Khi dự đoán trên dữ liệu mới, kết quả cuối cùng được tổng hợp từ tất cả các cây theo cơ chế majority voting cho bài toán phân loại. Bằng cách tổng hợp từ nhiều cây đa dạng, Random Forest giảm phương sai của dự đoán mà không làm tăng đáng kể bias.

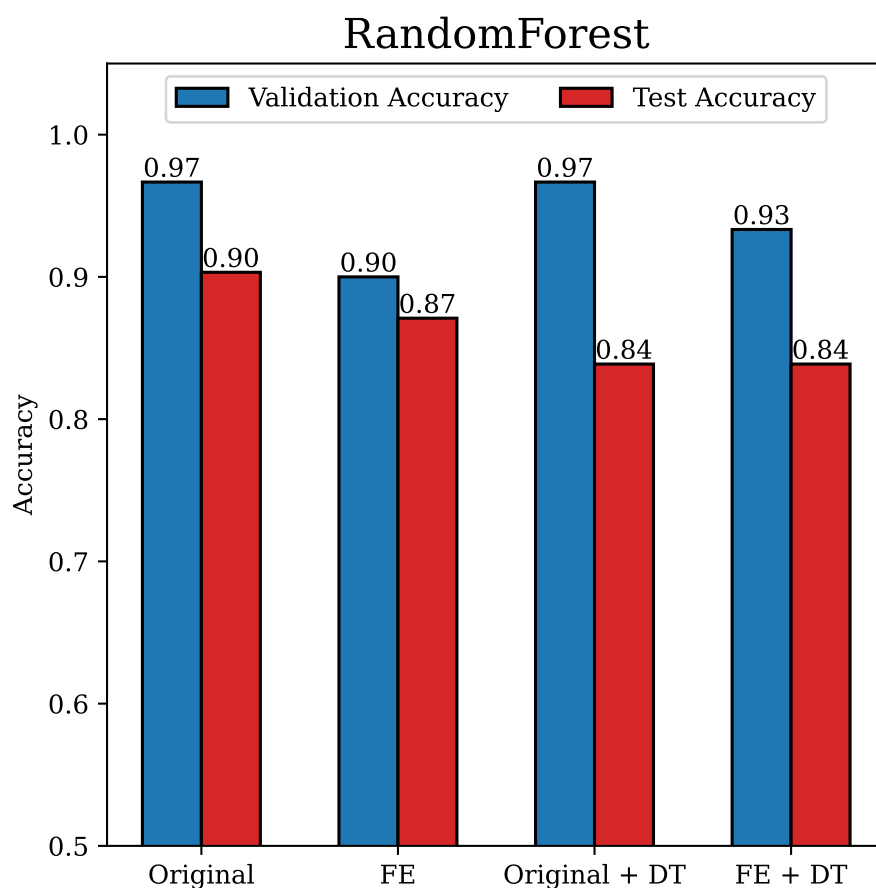
3.4.2. Triển khai

Random Forest

```
1 rf_results = run_benchmark(  
2     "Random Forest",  
3     make_model=lambda n: RandomForestClassifier(  
4         n_estimators=n, max_depth=5, min_samples_split=2,  
5         min_samples_leaf=1, max_features="sqrt", bootstrap=True,  
6         n_jobs=-1, random_state=SEED  
7     ),  
8     param_values=range(50, 501, 50),  
9     param_name="n_estimators"  
10 )  
11 plot_results(rf_results, "Random Forest")
```



Hình 13: Độ chính xác Cross-Validation của Random Forest tại các giá trị `n_estimators` khác nhau.

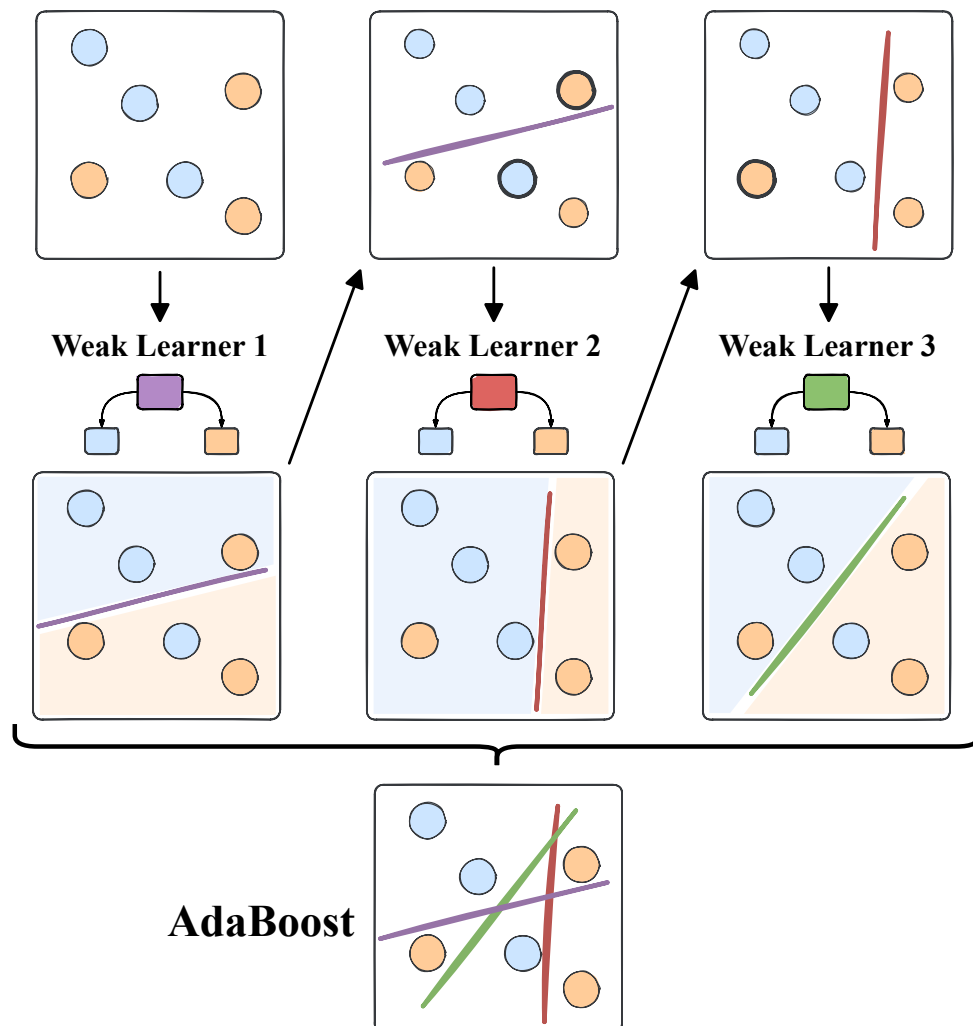


Hình 14: Hiệu năng của Random Forest trên bốn biến thể dữ liệu.

Ở đây cần lưu ý rằng `max_depth` được cố định bằng 5 trong quá trình dò `n_estimators`. Đây là một đơn giản hóa có chủ đích để giữ chi phí tính toán ở mức hợp lý, nhưng đồng thời cũng là một hạn chế: nếu quan hệ trong dữ liệu phức tạp hơn mức mà cây độ sâu 5 nắm bắt được, mô hình sẽ underfitting bất kể số cây tăng lên bao nhiêu.

3.5. AdaBoost

3.5.1. Giới thiệu



Hình 15: Minh họa thuật toán AdaBoost cho bài toán phân loại.

AdaBoost [12] hoạt động theo một quy trình lặp, chú trọng vào các điểm dữ liệu mà các mô hình trước đó đã phân loại sai:

- Ban đầu, mỗi điểm dữ liệu trong tập huấn luyện đều có trọng số như nhau.
- Một weak learner, thường là một cây quyết định rất nông (decision stump), được huấn luyện trên dữ liệu.
- Sau mỗi vòng, trọng số của các điểm dữ liệu được điều chỉnh. Các điểm bị phân loại sai được tăng trọng số, các điểm phân loại đúng giảm trọng số. Điều này buộc weak learner tiếp theo tập trung hơn vào các điểm khó.
- Quá trình lặp lại với các weak learner mới, mỗi mô hình tập trung vào lỗi của mô hình trước đó.

- Cuối cùng, các weak learner được kết hợp thành một mô hình tổng thể. Mỗi weak learner được gán một trọng số dựa trên hiệu năng của nó, mô hình có độ chính xác cao hơn sẽ có tiếng nói lớn hơn trong quyết định cuối cùng.

Khác với Random Forest vốn giảm phương sai, Boosting chủ yếu giảm bias bằng cách xây dựng dần một mô hình phức tạp từ nhiều mô hình yếu.

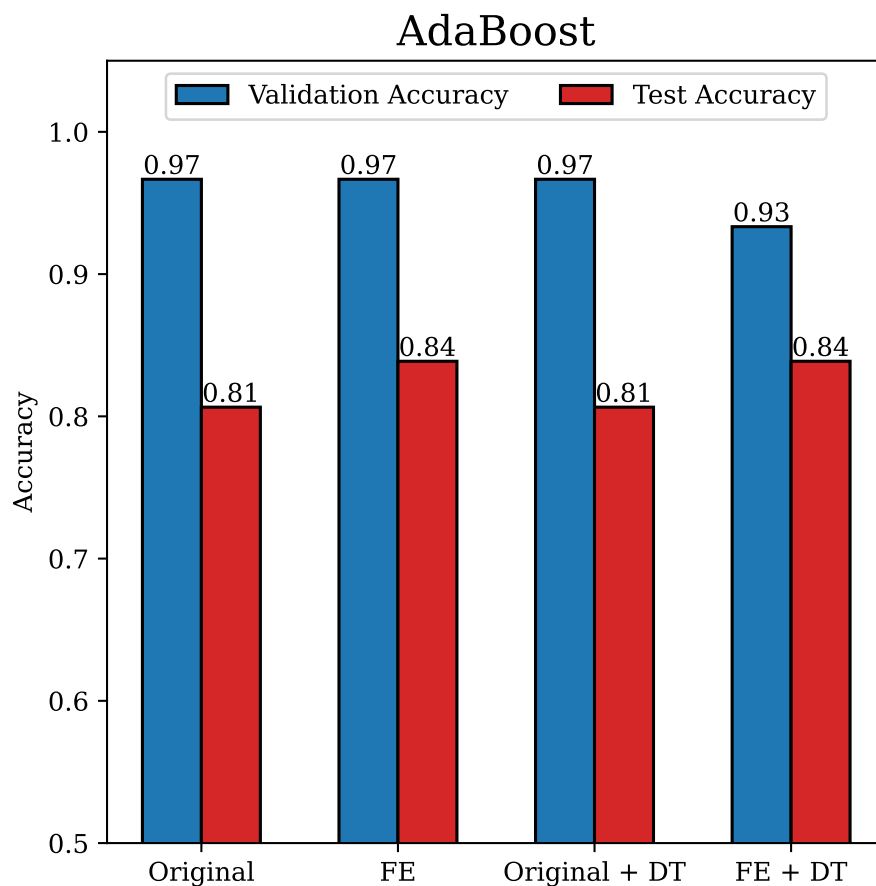
3.5.2. Triển khai

AdaBoost

```

1 ada_results = run_benchmark(
2     "AdaBoost",
3     make_model=lambda n: AdaBoostClassifier(
4         estimator=DecisionTreeClassifier(max_depth=1, random_state=SEED),
5         n_estimators=n, learning_rate=0.1, random_state=SEED
6     ),
7     param_values=range(50, 501, 50),
8     param_name="n_estimators"
9 )
10 plot_results(ada_results, "AdaBoost")

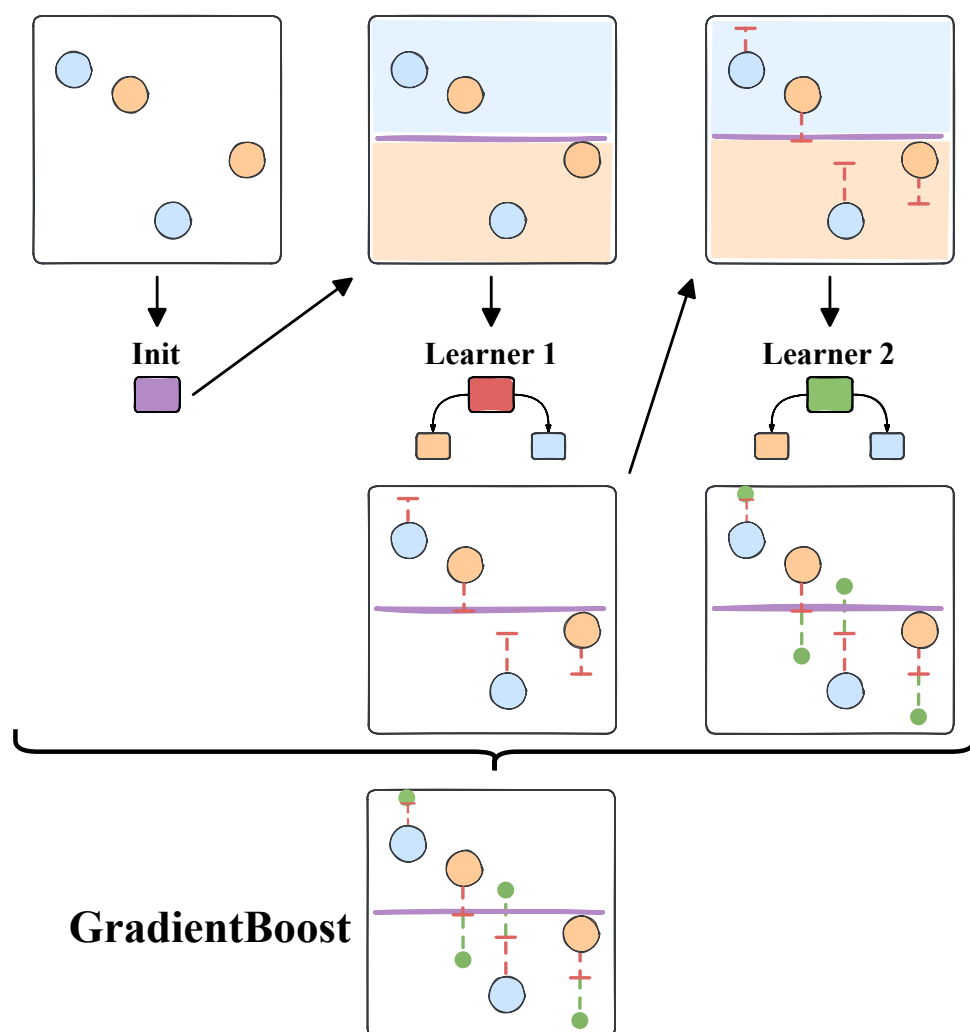
```



Hình 16: Hiệu năng của AdaBoost trên bốn biến thể dữ liệu.

3.6. Gradient Boosting

3.6.1. Giới thiệu



Hình 17: Minh họa thuật toán Gradient Boosting cho bài toán phân loại.

Gradient Boosting bắt đầu với một mô hình khởi tạo rất đơn giản, thường là một hằng số. Sau đó thuật toán lặp lại quá trình sau:

- Tại mỗi bước, tính phần dư (residual), tức là sự khác biệt giữa giá trị thực tế và giá trị dự đoán của mô hình hiện tại. Phần dư này chính là “lỗi” mà mô hình cần học cách sửa.
- Một mô hình mới, thường là một cây quyết định, được huấn luyện để dự đoán chính phần dư đó. Thay vì dự đoán kết quả cuối cùng, mô hình này tập trung vào việc sửa lỗi của mô hình trước.
- Mô hình mới được cộng vào mô hình tổng thể, nhân với một learning rate nhỏ để kiểm soát tốc độ học và tránh overfitting quá nhanh.

Điểm khác biệt cốt lõi so với AdaBoost nằm ở đối tượng mà mô hình kế tiếp học. AdaBoost điều chỉnh *trọng số của các mẫu*, buộc mô hình sau tập trung vào mẫu khó. Gradient Boosting huấn luyện mô hình sau để *trực tiếp dự đoán phần dư* của mô hình tổng hợp trước đó.

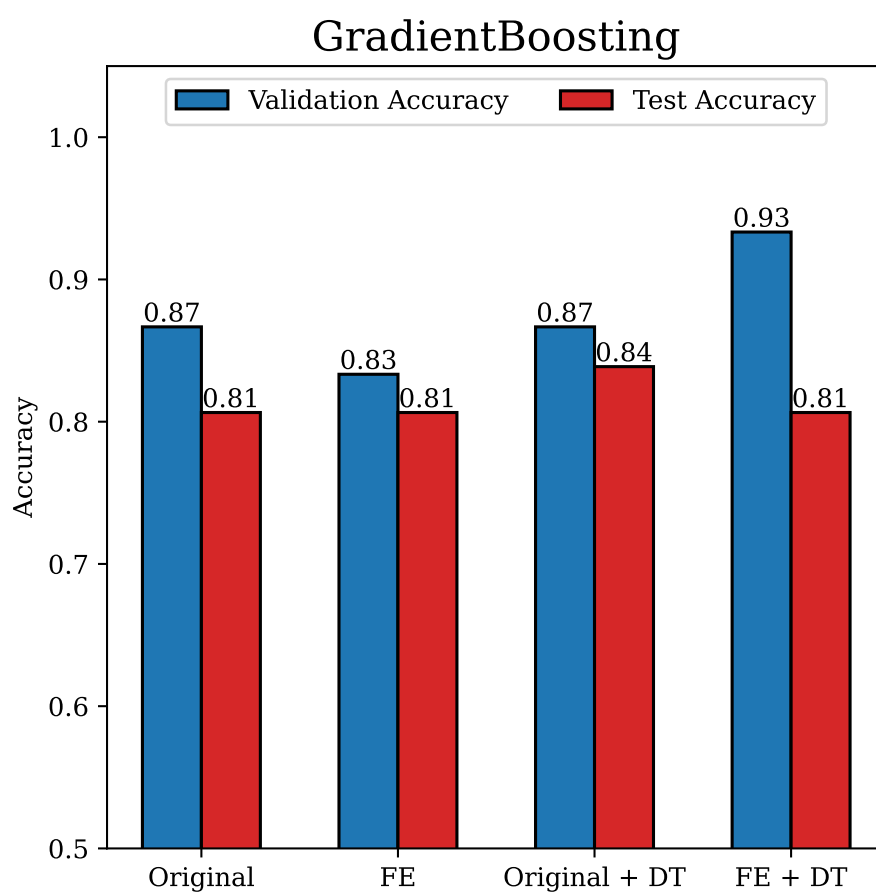
3.6.2. Triển khai

Gradient Boosting

```

1 gb_results = run_benchmark(
2     "Gradient Boosting",
3     make_model=lambda n: GradientBoostingClassifier(
4         n_estimators=n, learning_rate=0.1, max_depth=5,
5         subsample=1.0, random_state=SEED
6     ),
7     param_values=range(50, 501, 50),
8     param_name="n_estimators"
9 )
10 plot_results(gb_results, "Gradient Boosting")

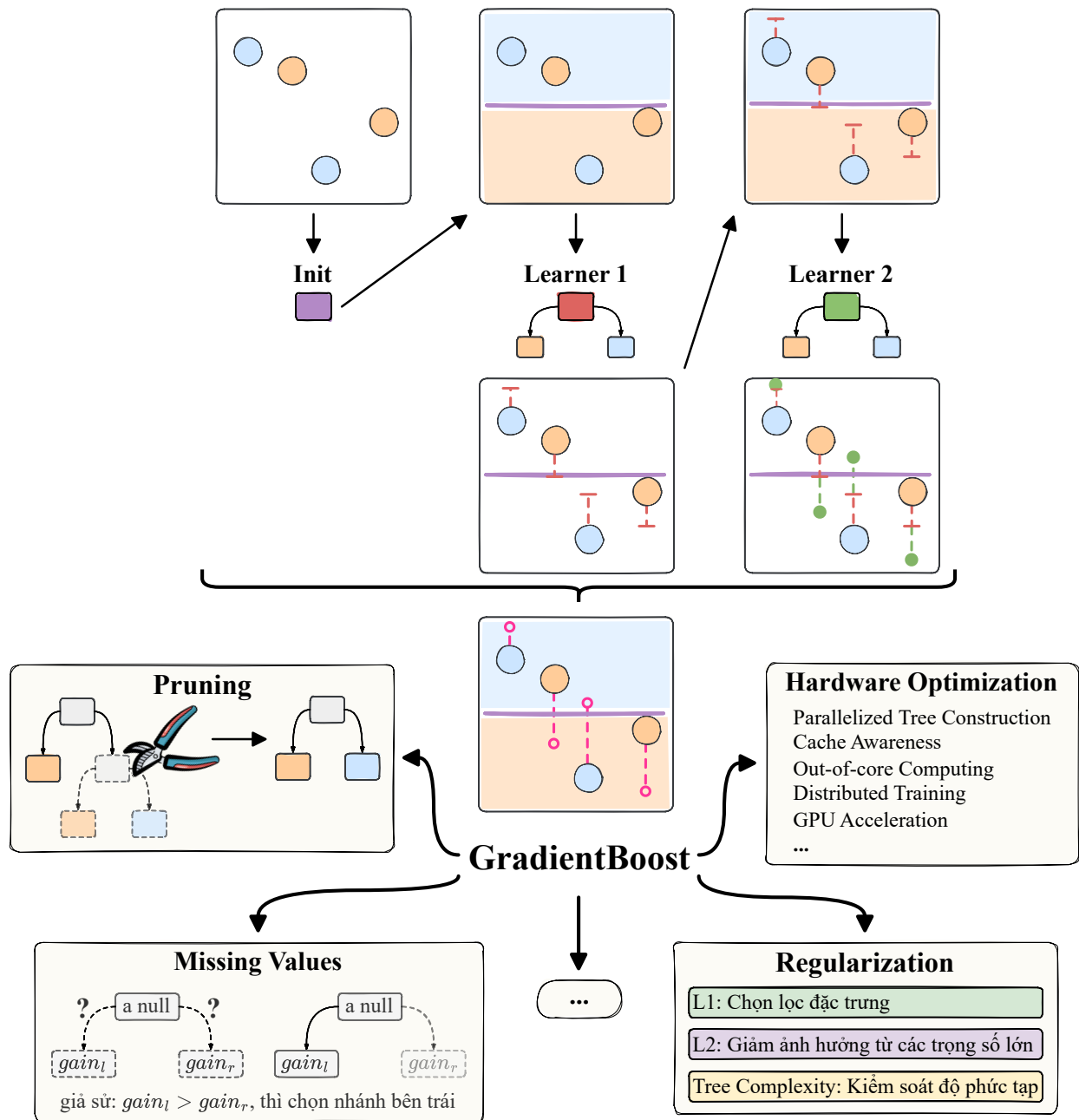
```



Hình 18: Hiệu năng của Gradient Boosting trên bốn biến thể dữ liệu.

3.7. XGBoost

3.7.1. Giới thiệu



Hình 19: Minh họa thuật toán XGBoost cho bài toán phân loại.

XGBoost [13] cải tiến từ Gradient Boosting bằng cách kết hợp nhiều kỹ thuật để tối ưu hóa hiệu năng và giảm thiểu overfitting. Các đặc điểm nổi bật gồm:

- Tối ưu hóa và song song hóa nhiều phép tính trong quá trình tìm điểm chia và xây dựng từng cây, đồng thời hỗ trợ GPU acceleration để tăng tốc huấn luyện trên các bộ dữ liệu lớn.
- Tự động xử lý missing values bằng cách học hướng phân nhánh mặc định ngay cả khi dữ liệu bị thiếu.

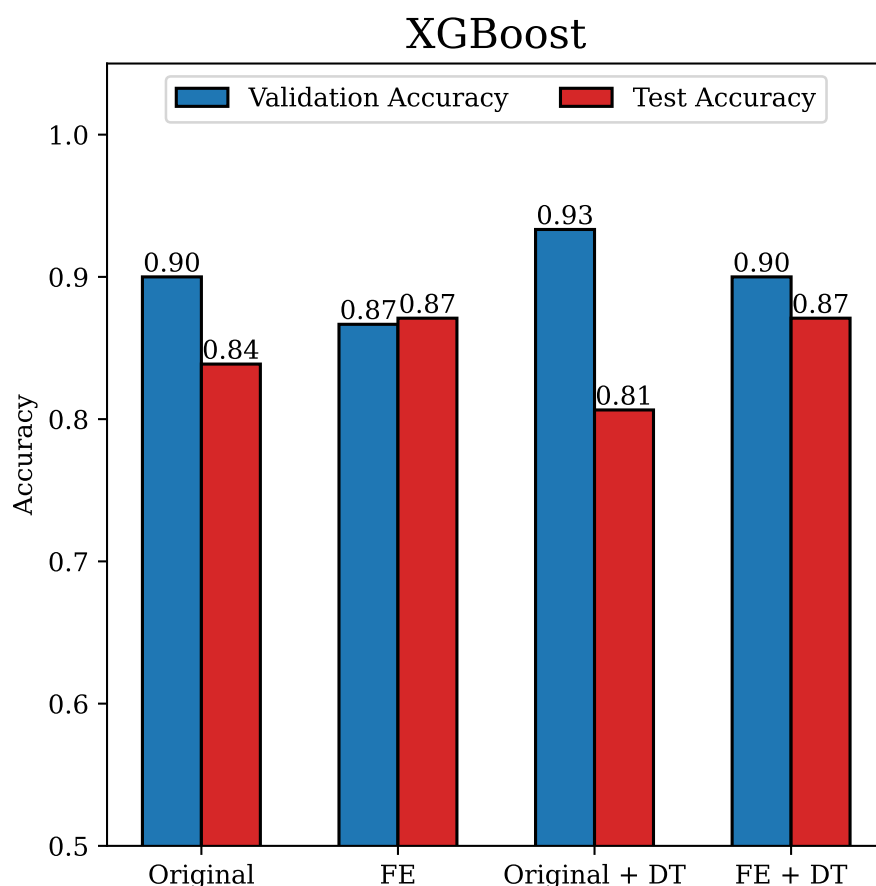
- Áp dụng regularization L1 và L2 để kiểm soát độ phức tạp của mô hình. Pruning cũng được sử dụng để loại bỏ các nhánh không cần thiết.

Về cơ bản, XGBoost vẫn xây dựng một chuỗi cây quyết định tuần tự, mỗi cây mới cố gắng sửa lỗi của các cây trước đó. Điểm khác biệt nằm ở hàm mục tiêu có thêm thành phần regularization và ở các tối ưu hóa kỹ thuật ở tầng cài đặt.

3.7.2. Triển khai

XGBoost

```
1 USE_GPU = False
2 xgb_results = run_benchmark(
3     "XGBoost",
4     make_model=lambda n: XGBClassifier(
5         n_estimators=n, learning_rate=0.1, max_depth=5, subsample=1.0,
6         objective="binary:logistic", eval_metric="logloss",
7         tree_method="hist", device="cuda" if USE_GPU else "cpu",
8         n_jobs=-1, verbosity=0, random_state=SEED
9     ),
10    param_values=range(50, 501, 50),
11    param_name="n_estimators"
12 )
13 plot_results(xgb_results, "XGBoost")
```

Hình 20: Hiệu năng của XGBoost trên bốn biến thể dữ liệu.

4. Giải thích mô hình bằng TreeSHAP

4.1. Động lực

Các mô hình ensemble dựa trên cây như Random Forest và Gradient Boosting thường đạt hiệu năng tốt trên dữ liệu dạng bảng, nhưng khó diễn giải hơn một Decision Tree đơn lẻ. Trong bài toán liên quan đến dữ liệu y tế, bên cạnh việc biết mô hình dự đoán bệnh nhân “có bệnh” hay “không bệnh”, ta còn muốn hiểu những đặc trưng nào đã đóng góp nhiều nhất vào dự đoán đó và chúng làm đầu ra của mô hình tăng hay giảm.

Để giải quyết vấn đề này, chúng ta sử dụng **TreeSHAP**, một phương pháp giải thích dành cho các mô hình dựa trên cây được phát triển từ SHAP [7, 8]. Ý tưởng của SHAP bắt nguồn từ Shapley value trong lý thuyết trò chơi, trong đó mỗi đặc trưng được xem như một “người chơi” cùng đóng góp vào kết quả dự đoán của mô hình.

4.2. TreeSHAP

TreeSHAP khai thác trực tiếp cấu trúc của các cây quyết định để tính SHAP values hiệu quả cho Decision Tree và các mô hình ensemble của cây. Với một mẫu đầu vào x , dự đoán của mô hình có thể được biểu diễn dưới dạng cộng:

$$f(x) = \phi_0 + \sum_{j=1}^M \phi_j,$$

trong đó ϕ_0 là giá trị dự đoán cơ sở của mô hình và ϕ_j là mức đóng góp của đặc trưng thứ j vào dự đoán của mẫu đang xét. Nếu ϕ_j có giá trị dương, đặc trưng đó làm đầu ra của mô hình tăng so với giá trị cơ sở. Ngược lại, nếu ϕ_j có giá trị âm, đặc trưng đó làm đầu ra giảm.

Nhờ đó, TreeSHAP có thể được sử dụng ở hai mức độ. Ở mức **toàn cục**, ta có thể tổng hợp SHAP values trên nhiều bệnh nhân để xác định những đặc trưng có ảnh hưởng lớn đến dự đoán của mô hình. Ở mức **cục bộ**, TreeSHAP cho phép phân tích một bệnh nhân cụ thể để thấy từng đặc trưng đã đóng góp như thế nào vào dự đoán cuối cùng.

4.3. Triển khai TreeSHAP

Trong phần này, TreeSHAP được áp dụng cho mô hình Random Forest đã huấn luyện trên bộ Original. Đây là cấu hình đạt kết quả tốt nhất của Random Forest trong thí nghiệm fixed-split, với test accuracy đạt 90,32%. Mô hình này đã được lưu trong biến `rf_model_original` trước khi tiếp tục các thí nghiệm trên những biến thể dữ liệu khác.

Cài đặt và tính SHAP values

```

1 !pip -q install shap
2
3 import shap
4 import numpy as np
5 import pandas as pd
6 import matplotlib.pyplot as plt
7
8 X_shap = X_test.copy()
9 y_shap = y_test.copy()
10 model_shap = rf_model_original
11
12 explainer = shap.TreeExplainer(model_shap)
13 shap_values = explainer(X_shap)
14
15 print("Raw SHAP values shape:",
16       shap_values.values.shape)
17
18 if shap_values.values.ndim == 3:
19     shap_disease = shap_values[:, :, 1]
20 else:
21     shap_disease = shap_values
22
23 print("Disease-class SHAP shape:",
24       shap_disease.values.shape)
25 print("Number of samples:",
26       shap_disease.shape[0])
27 print("Number of features:",
28       shap_disease.shape[1])

```

Với bài toán phân loại nhị phân, Random Forest trả về xác suất cho hai lớp gồm lớp 0 (không bệnh) và lớp 1 (có bệnh). Vì mục tiêu của phần giải thích là phân tích dự đoán “có bệnh”,

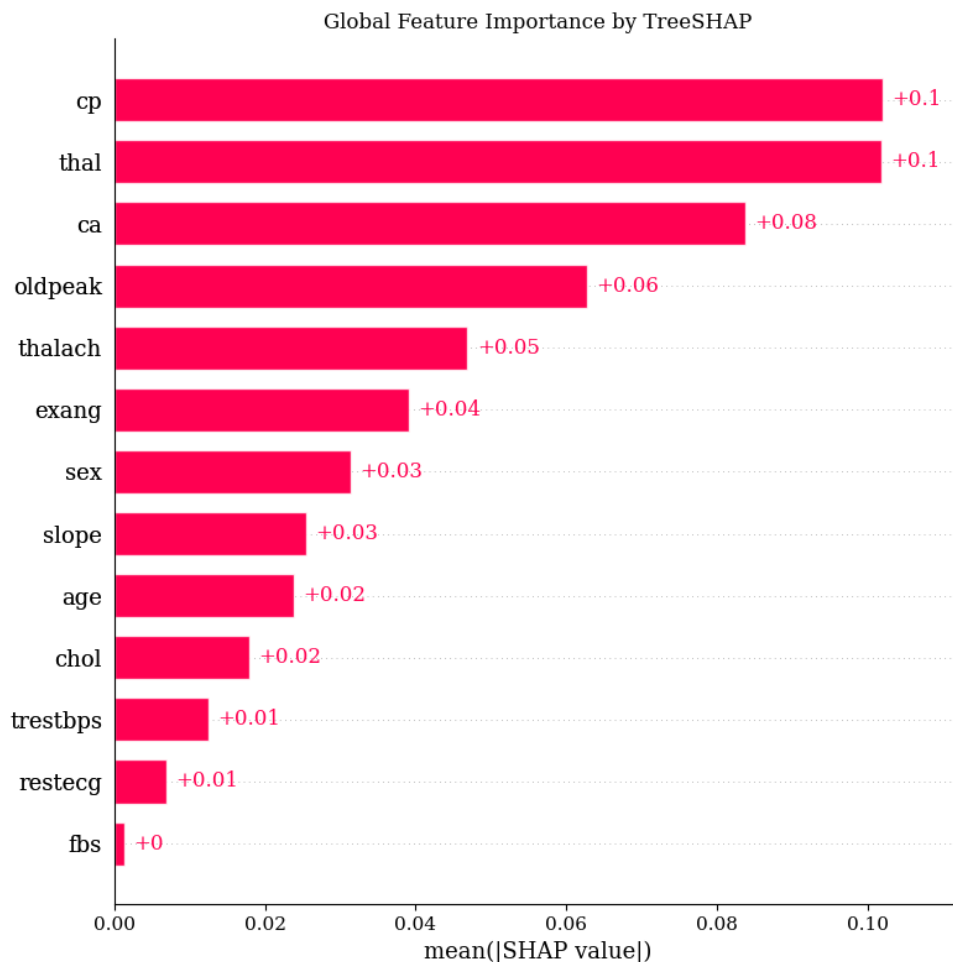
SHAP values của lớp 1 được sử dụng trong các phân tích tiếp theo.

4.4. Giải thích toàn cục

Để đánh giá mức độ đóng góp của các đặc trưng trên toàn bộ tập test, TreeSHAP tổng hợp trị tuyệt đối của SHAP values trên nhiều bệnh nhân. Đặc trưng có mean absolute SHAP value lớn hơn là đặc trưng tạo ra thay đổi lớn hơn đối với đầu ra của mô hình trên tập mẫu được phân tích.

Global feature importance bằng TreeSHAP

```
1 shap.plots.bar(
2     shap_disease,
3     max_display=min(13, X_shap.shape[1]),
4     show=False
5 )
6
7 plt.title("Global Feature Importance by TreeSHAP")
8 plt.tight_layout()
9 plt.show()
```



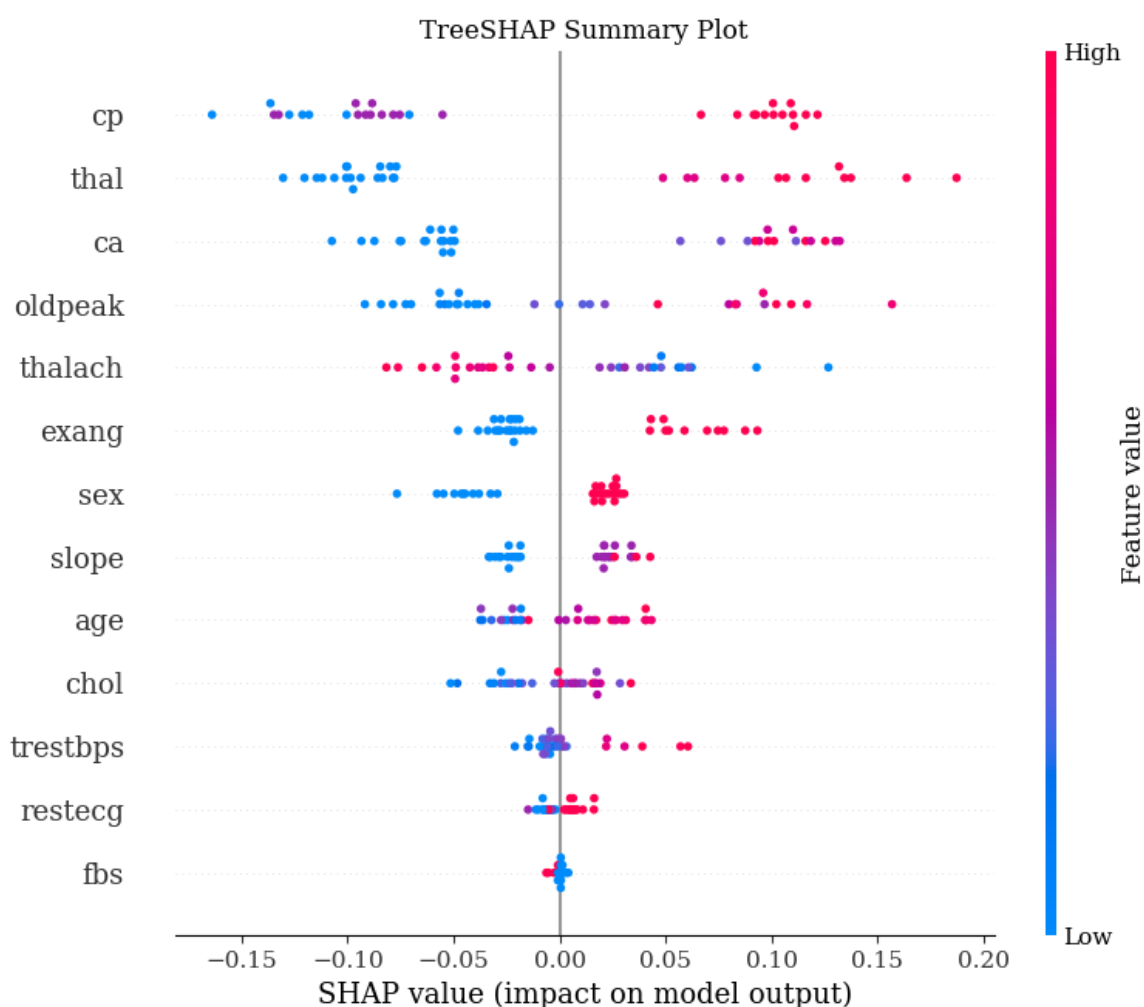
Hình 21: Mức độ quan trọng toàn cục của các đặc trưng trong Random Forest dựa trên mean absolute SHAP values.

4.5. Hướng đóng góp của đặc trưng

Biểu đồ global importance cho biết đặc trưng nào có mức đóng góp lớn nhưng không thể hiện đầy đủ hướng đóng góp của từng mẫu. Vì vậy, biểu đồ beeswarm được sử dụng để quan sát đồng thời độ lớn của SHAP value, hướng đóng góp và giá trị của đặc trưng trên nhiều bệnh nhân trong tập test.

SHAP beeswarm plot

```
1 shap.plots.beeswarm(
2     shap_disease,
3     max_display=min(13, X_shap.shape[1]),
4     show=False
5 )
6
7 plt.title("TreeSHAP Summary Plot")
8 plt.tight_layout()
9 plt.show()
```



Hình 22: Phân bố SHAP values của các đặc trưng trên tập test.

Trong biểu đồ beeswarm, mỗi điểm tương ứng với một bệnh nhân. Vị trí của điểm trên trục

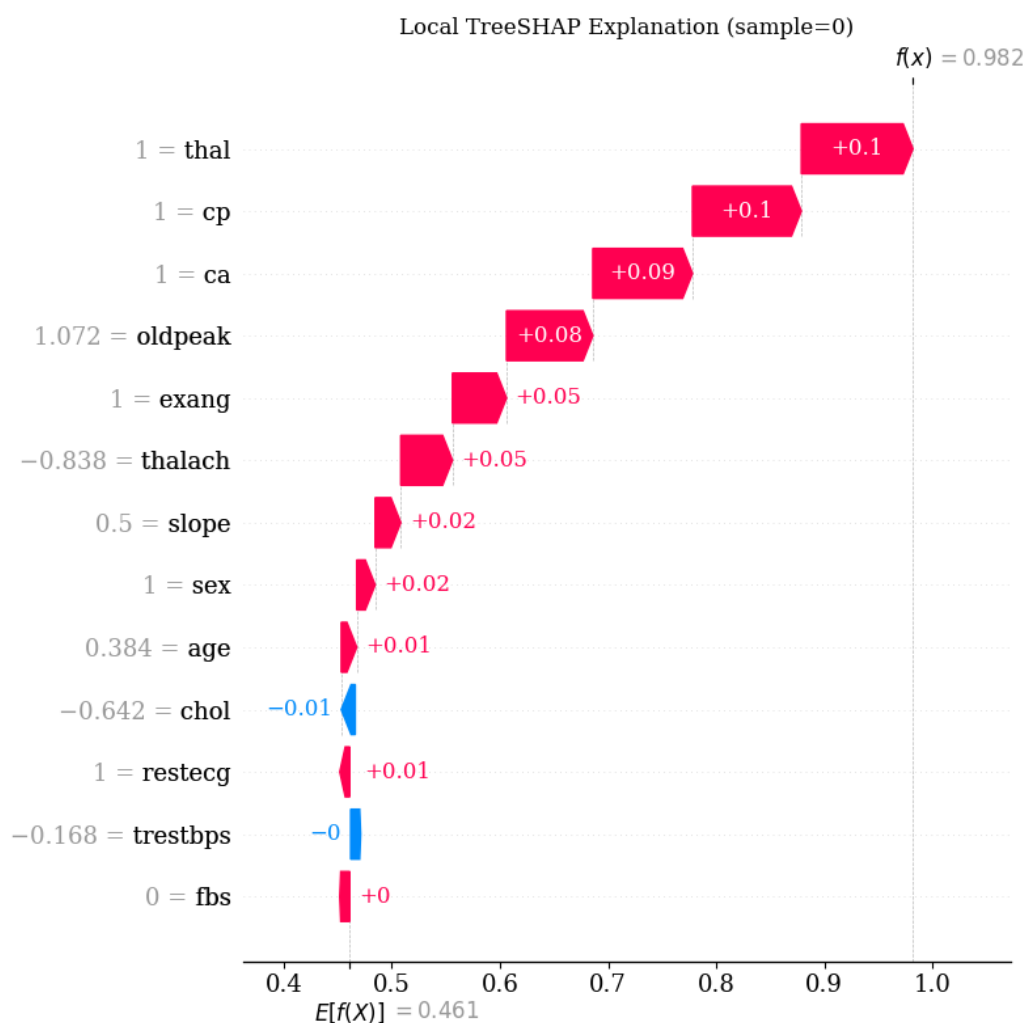
SHAP cho biết đặc trưng đang làm đầu ra của mô hình tăng hay giảm so với giá trị cơ sở, trong khi màu sắc biểu diễn giá trị thấp hoặc cao của đặc trưng đó. Nhờ vậy, biểu đồ này cho phép quan sát cả mức độ quan trọng và cách một đặc trưng đóng góp vào các dự đoán khác nhau.

4.6. Giải thích một dự đoán cụ thể

Bên cạnh khả năng giải thích toàn cục, TreeSHAP có thể cho phép phân tích từng bệnh nhân riêng lẻ. Trong ví dụ dưới đây, chúng ta tìm các bệnh nhân thực sự thuộc lớp có bệnh và được Random Forest dự đoán đúng, sau đó lựa chọn trường hợp mà mô hình dự đoán lớp có bệnh với xác suất cao nhất. Nếu không tìm thấy trường hợp như vậy, mẫu có xác suất dự đoán bệnh cao nhất được sử dụng thay thế.

Lựa chọn bệnh nhân và giải thích cục bộ

```
1 test_pred = model_shap.predict(X_shap)
2 test_prob = model_shap.predict_proba(X_shap)[: , 1]
3
4 correct_positive = np.where((y_shap.to_numpy() == 1) & (test_pred == 1))[0]
5
6 if len(correct_positive) > 0:
7     local_id = correct_positive[np.argmax(test_prob[correct_positive])]
8 else:
9     local_id = int(np.argmax(test_prob))
10
11 print("Selected sample:", local_id)
12 print("Ground truth:", int(y_shap.iloc[local_id]))
13 print("Prediction:", int(test_pred[local_id]))
14 print("Disease probability:", f"{test_prob[local_id]:.4f}")
15 display(X_shap.iloc[[local_id]])
16
17 shap.plots.waterfall(
18     shap_disease[local_id],
19     max_display=min(13, X_shap.shape[1]),
20     show=False
21 )
22
23 plt.title(f"Local TreeSHAP Explanation (sample={local_id})")
24 plt.tight_layout()
25 plt.show()
```



Hình 23: Giải thích TreeSHAP cho một bệnh nhân được Random Forest dự đoán thuộc lớp có bệnh.

Waterfall plot bắt đầu từ giá trị dự đoán cơ sở của mô hình và thể hiện cách từng đặc trưng làm đầu ra tăng hoặc giảm để hình thành dự đoán cuối cùng cho bệnh nhân đang xét. Khác với global feature importance, cách biểu diễn này cho phép quan sát trực tiếp vai trò của từng đặc trưng trong một dự đoán cụ thể.

5. Kết quả thực nghiệm

5.1. Thiết lập thực nghiệm

Mọi mô hình được đánh giá trên cùng một phép chia train/val/test cố định (80:10:10, có phân tầng theo nhân) và cùng bốn biến thể dữ liệu. Siêu tham số được chọn bằng Stratified 5-fold Cross-Validation *chỉ trên tập train*, tập validation dùng để đối chiếu, tập test chỉ được sử dụng một lần ở bước báo cáo cuối cùng.

Cần nêu rõ một giới hạn của giao thức này. Với 303 mẫu, tập test chỉ có khoảng 30 bệnh nhân, nghĩa là mỗi mẫu bị phân loại sai làm thay đổi accuracy khoảng 3,3 điểm phần trăm. Chênh lệch nhỏ giữa các mô hình trong Bảng 2 do đó không mang ý nghĩa thống kê và không nên được diễn giải như bằng chứng về việc mô hình này tốt hơn mô hình kia. Để có kết luận chắc chắn hơn, phần dưới bổ sung một đánh giá bằng repeated stratified cross-validation trên toàn bộ dữ liệu, báo cáo trung bình và độ lệch chuẩn.

Đánh giá bổ sung bằng repeated stratified CV

```
1 from sklearn.model_selection import RepeatedStratifiedKFold, cross_val_score
2
3 def robust_eval(make_model, best_param, X, y, n_repeats=10):
4     cv = RepeatedStratifiedKFold(
5         n_splits=5, n_repeats=n_repeats, random_state=SEED
6     )
7     scores = cross_val_score(
8         make_model(best_param), X, y, cv=cv, scoring="f1", n_jobs=-1
9     )
10    return scores.mean(), scores.std()
```

5.2. Bảng tổng hợp

Bảng 2 ghi nhận độ chính xác của các phương pháp trên tập validation và test của từng biến thể dữ liệu. Đây là cách trình bày thường thấy trong các bài báo khoa học, cung cấp một góc nhìn toàn diện về khả năng của tất cả các phương pháp được khảo sát. Các phương pháp được sắp xếp theo thứ tự trình bày trong bài, với hai baseline không thuộc họ cây đặt ở nhóm trên cùng để dễ đối chiếu.

Bảng 2: Tổng hợp kết quả thực nghiệm. FE là viết tắt của “feature engineering”; DT là viết tắt của “Decision Tree” (dùng để chọn đặc trưng). Giá trị in đậm là kết quả cao nhất theo từng cột.

Model	Val				Test			
	Original	FE	Original+DT	FE+DT	Original	FE	Original+DT	FE+DT
<i>Baseline không thuộc họ cây</i>								
Naive Bayes	.90	.90	.93	.93	.84	.84	.84	.84
KNN	.90	.90	.97	.87	.84	.84	.87	.84
<i>Họ cây</i>								
Decision Tree	.93	.93	.93	.93	.81	.81	.81	.81
Random Forest	.97	.90	.97	.93	.90	.87	.84	.84
AdaBoost	.97	.97	.97	.93	.81	.84	.81	.84
Gradient Boosting	.87	.83	.87	.93	.81	.81	.84	.81
XGBoost	.90	.87	.93	.90	.84	.87	.81	.87

5.3. Phân tích

Dựa vào Bảng 2, có thể rút ra một số nhận xét.

Về hiệu năng tổng thể. Độ chính xác cao nhất đạt được là 97% trên tập validation và 90% trên tập test. Các phương pháp thuộc nhóm ensemble, cụ thể là Random Forest và AdaBoost, cho kết quả validation ổn định nhất trên cả bốn biến thể dữ liệu. Trên tập test, Random Forest với bộ Original cho kết quả tốt nhất.

Về khoảng cách giữa validation và test. Hầu hết mô hình đều có accuracy trên test thấp hơn trên validation từ 6 đến 16 điểm phần trăm. AdaBoost là trường hợp rõ nhất, với 97% trên validation nhưng chỉ 81% trên test. Đây là dấu hiệu của việc siêu tham số đã được chọn quá khớp với một tập validation nhỏ, chứ không hẳn là bằng chứng về overfitting của bản thân mô hình.

Về tác động của tiền xử lý. Việc áp dụng feature engineering và feature selection không mang lại cải thiện nhất quán. Điều này phù hợp với đặc thù của các mô hình cây: chúng vốn đã thực hiện việc chọn đặc trưng một cách ngầm định trong quá trình xây cây, và không nhạy với scaling. Ngược lại, hai baseline không thuộc họ cây (đặc biệt là KNN, vốn phụ thuộc trực tiếp vào thang đo khoảng cách) hưởng lợi rõ hơn từ bước rút gọn đặc trưng bằng Decision Tree.

Về so sánh cây và không thuộc họ cây. Trên bộ dữ liệu này, khoảng cách giữa họ cây và các baseline không thuộc họ cây không lớn. Đây là điều dự đoán được: với 303 mẫu và 13 đặc trưng, không gian bài toán đủ nhỏ để các mô hình đơn giản cũng nắm bắt được phần lớn tín hiệu. Ưu thế của các mô hình cây trên dữ liệu bảng thường thể hiện rõ hơn khi số mẫu và số đặc trưng tăng lên.

Khuyến nghị. Với bài toán này, Random Forest là lựa chọn hợp lý nhất: hiệu năng cao và ổn định, ít siêu tham số cần dò, chi phí huấn luyện thấp và vẫn cung cấp feature importance để đối chiếu với kiến thức y khoa. Các phương pháp Boosting không cho thấy lợi thế đủ rõ để bù lại chi phí dò siêu tham số cao hơn trên một bộ dữ liệu nhỏ như vậy.

6. Xây dựng demo

Để nâng cao tính hoàn thiện và trực quan của dự án, phần này xây dựng một bản demo với giao diện đơn giản bằng Streamlit. Bản demo minh họa trực tiếp hiệu quả của mô hình, đồng thời hỗ trợ quá trình kiểm thử.

Google Colab không được thiết kế để lưu trữ ứng dụng web, nên để chạy và truy cập giao diện Streamlit từ Colab cần tạo một đường hầm (tunnel). Quy trình gồm ba bước: viết mã ứng dụng vào `app.py`, cài đặt phụ thuộc và khởi chạy Streamlit, cuối cùng dùng Cloudflared để mở tunnel và nhận một URL công khai.

6.1. Cài đặt ứng dụng

Trong môi trường Colab, dùng magic command `%%writefile` để ghi toàn bộ code ứng dụng vào một tệp Python. Phần đầu khai báo thư viện và cấu hình trang.

app.py - Khai báo và cấu hình

```
1 %%writefile app.py
2 import streamlit as st
3 import pandas as pd
4 import matplotlib.pyplot as plt
5
6 from sklearn.pipeline import Pipeline
7 from sklearn.impute import SimpleImputer
8 from sklearn.compose import ColumnTransformer
9 from sklearn.preprocessing import OneHotEncoder
10 from sklearn.model_selection import GridSearchCV
11 from sklearn.ensemble import RandomForestClassifier
12
13 st.set_page_config(page_title="Heart Disease Prediction", layout="wide")
14
15 URL = ("https://archive.ics.uci.edu/ml/machine-learning-databases/"
16       "heart-disease/processed.cleveland.data")
17 COLUMNS = ["age", "sex", "cp", "trestbps", "chol", "fbs", "restecg",
18            "thalach", "exang", "oldpeak", "slope", "ca", "thal", "target"]
19 NUMERIC = ["age", "trestbps", "chol", "thalach", "oldpeak"]
20 CATEGORICAL = ["sex", "cp", "fbs", "restecg", "exang", "slope", "ca", "thal"]
```

Phần tiếp theo là logic cốt lõi: tải dữ liệu, xây dựng pipeline hoàn chỉnh và dùng `GridSearchCV` để tự động tìm siêu tham số tốt nhất. Mô hình được chọn là Random Forest, thống nhất với kết luận ở phần 5.. Decorator `@st.cache_resource` đảm bảo quá trình tốn thời gian này chỉ chạy một lần khi ứng dụng khởi động.

app.py - Huấn luyện mô hình

```
1 @st.cache_resource
2 def find_best_model():
3     df = pd.read_csv(URL, header=None, names=COLUMNS, na_values="?")
4     df = df.dropna()
5     df["target"] = (df["target"] > 0).astype(int)
6
7     X = df[NUMERIC + CATEGORICAL]
```

```

8     y = df["target"]
9
10    num_pipe = Pipeline([("imputer", SimpleImputer(strategy="median"))])
11    cat_pipe = Pipeline([
12        ("imputer", SimpleImputer(strategy="most_frequent")),
13        ("ohe", OneHotEncoder(handle_unknown="ignore", sparse_output=False))
14    ])
15    preprocessor = ColumnTransformer([
16        ("num", num_pipe, NUMERIC),
17        ("cat", cat_pipe, CATEGORICAL)
18    ])
19    pipeline = Pipeline([
20        ("preprocessor", preprocessor),
21        ("classifier", RandomForestClassifier(random_state=42, n_jobs=-1))
22    ])
23
24    param_grid = {
25        "classifier__n_estimators": [100, 200, 300],
26        "classifier__max_depth": [3, 5, 7],
27    }
28    grid_search = GridSearchCV(
29        pipeline, param_grid, cv=5, scoring="accuracy", n_jobs=-1
30    )
31    grid_search.fit(X, y)
32    return grid_search
33
34
35 model = find_best_model()

```

Giao diện nhập liệu được đặt trong một form ở thanh bên, dùng `st.slider` cho biến số và `st.selectbox` cho biến phân loại.

app.py - Giao diện người dùng

```

1 st.title("Heart Disease Prediction App")
2 st.write("Enter patient data in the sidebar to get a prediction.")
3
4 best_params = model.best_params_
5 st.sidebar.info(f"Best params: {best_params}")
6
7 with st.sidebar.form("input_form"):
8     st.header("Patient Parameters")
9     age = st.slider("Age", 20, 80, 50)
10    sex = st.selectbox("Gender", [("Male", 1), ("Female", 0)],
11                      format_func=lambda x: x[0])[1]
12    cp = st.selectbox("Chest Pain Type",
13                    [("Typical Angina", 1), ("Atypical Angina", 2),
14                    ("Non-anginal Pain", 3), ("Asymptomatic", 4)],
15                    format_func=lambda x: x[0])[1]
16    trestbps = st.slider("Resting Blood Pressure (mmHg)", 90, 200, 120)
17    chol = st.slider("Serum Cholesterol (mg/dl)", 120, 570, 240)
18    fbs = st.selectbox("Fasting Blood Sugar > 120 mg/dl",
19                    [("False", 0), ("True", 1)],
20                    format_func=lambda x: x[0])[1]
21    restecg = st.selectbox("Resting ECG",

```

```

22         [("Normal", 0), ("ST-T Abnormality", 1),
23          ("LV Hypertrophy", 2)],
24         format_func=lambda x: x[0])[1]
25 thalach = st.slider("Max Heart Rate Achieved", 70, 210, 150)
26 exang = st.selectbox("Exercise-induced Angina",
27                     [("No", 0), ("Yes", 1)],
28                     format_func=lambda x: x[0])[1]
29 oldpeak = st.slider("ST Depression", 0.0, 6.2, 1.0, step=0.1)
30 slope = st.selectbox("Slope of Peak ST",
31                     [("Upsloping", 1), ("Flat", 2), ("Downsloping", 3)],
32                     format_func=lambda x: x[0])[1]
33 ca = st.slider("Major Vessels colored by Fluoroscopy", 0, 3, 0)
34 thal = st.selectbox("Thalassemia",
35                    [("Normal", 3), ("Fixed Defect", 6),
36                    ("Reversible Defect", 7)],
37                    format_func=lambda x: x[0])[1]
38 submitted = st.form_submit_button("Predict")

```

Cuối cùng là phần dự đoán và trực quan hóa feature importance. Lưu ý rằng model là một đối tượng GridSearchCV, nên để truy cập các thành phần bên trong pipeline cần đi qua thuộc tính `best_estimator_`.

app.py - Dự đoán và trực quan hóa

```

1 if submitted:
2     input_data = pd.DataFrame([
3         "age": age, "sex": sex, "cp": cp, "trestbps": trestbps,
4         "chol": chol, "fbs": fbs, "restecg": restecg, "thalach": thalach,
5         "exang": exang, "oldpeak": oldpeak, "slope": slope,
6         "ca": ca, "thal": thal
7     ])
8     prob_disease = model.predict_proba(input_data)[0][1]
9
10    st.subheader("Prediction Result")
11    if prob_disease > 0.5:
12        st.error(f"Disease Likely - Probability: {prob_disease:.2%}")
13    else:
14        st.success(
15            f"Disease Unlikely - Probability of No Disease: "
16            f"{(1 - prob_disease):.2%}"
17        )
18
19    st.subheader("Feature Importance")
20    best_pipeline = model.best_estimator_
21    preprocessor = best_pipeline.named_steps["preprocessor"]
22    classifier = best_pipeline.named_steps["classifier"]
23
24    try:
25        feature_names = preprocessor.get_feature_names_out()
26        fi = pd.Series(
27            classifier.feature_importances_, index=feature_names
28        ).sort_values(ascending=False).head(15)
29
30        fig, ax = plt.subplots(figsize=(8, 6))

```

```
31 ax.barh(fi.index[::-1], fi.values[::-1])
32 ax.set_xlabel("Importance")
33 st.pyplot(fig)
34 except Exception as e:
35     st.warning(f"Could not display feature importance. Error: {e}")
```

6.2. Cài đặt môi trường và khởi chạy

[Cài đặt Streamlit và Cloudflared]

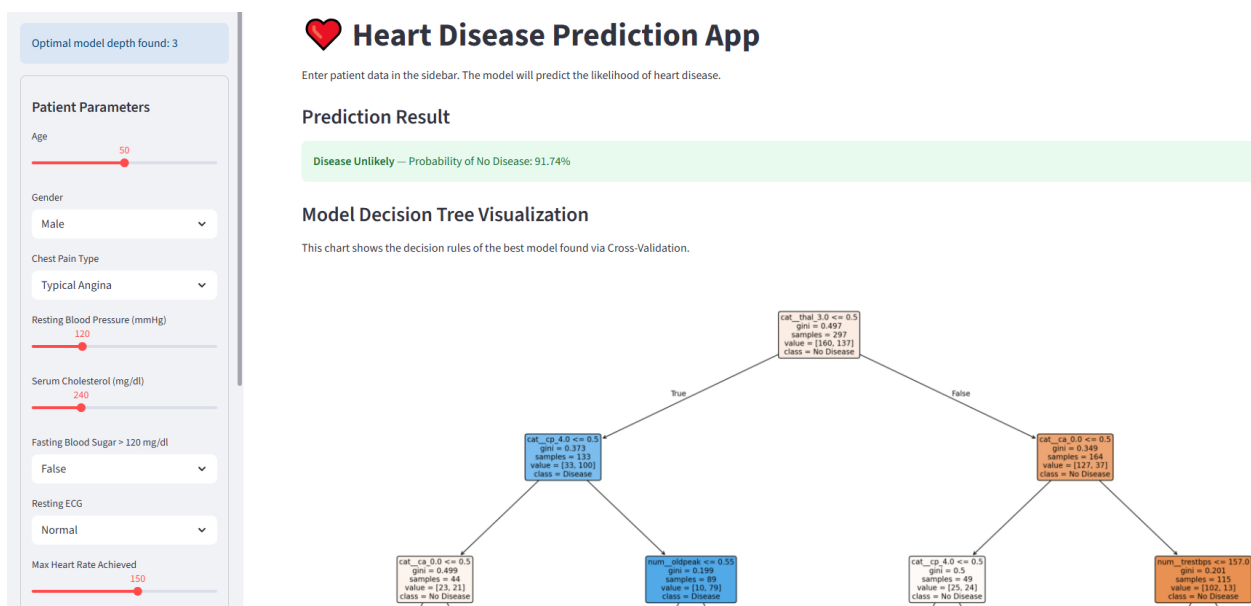
```
1 pip -q install streamlit
2
3 wget -q https://github.com/cloudflare/cloudflared/releases/latest/download/
   cloudflared-linux-amd64 -O cloudflared
4 chmod +x cloudflared
```

[Khởi chạy server và mở tunnel]

```
1 streamlit run app.py \
2   --server.port 8501 \
3   --server.address 0.0.0.0 \
4   --server.headless true \
5   --server.enableCORS false \
6   --server.enableXsrfProtection false &>/content/logs.txt &
7
8 ./cloudflared tunnel --url http://localhost:8501 --no-autoupdate
```

Kết quả và cách truy cập

Sau khi chạy cell cuối cùng, tìm trong output một đường link có dạng **https://....trycloudflare.com**. Đây là địa chỉ công khai của ứng dụng Streamlit. Nhấp vào link để mở ứng dụng trong một tab mới.



Hình 24: Giao diện của bản demo.

⚠ Cảnh báo

Toàn bộ nội dung dự án này phục vụ mục đích học tập. Các mô hình được xây dựng trên một bộ dữ liệu nhỏ, thu thập từ một cơ sở y tế duy nhất trong thập niên 1980, và không được kiểm định lâm sàng. Kết quả dự đoán không có giá trị chẩn đoán y khoa và không được sử dụng để thay thế ý kiến của bác sĩ.

7. Tài liệu tham khảo

- [1] PCR, *World Heart Day 2023: Reducing the burden of cardiovascular disease globally: Beyond stents and balloons!*, en. [Online]. Available: <https://www.pcronline.com/News/Whats-new-on-PCRonline/2023/World-Heart-Day-2023-Reducing-burden-cardiovascular-disease-globally-beyond-stents-balloons>.
- [2] *Cardiovascular Disease / NCD Alliance*, en, Oct. 2023. [Online]. Available: <https://ncdalliance.org/explore-ncds/ncds/cardiovascular-disease>.
- [3] World Heart Federation, *World Heart Report 2023: Full Report*, en-GB, World Heart Federation, 2023. [Online]. Available: <https://world-heart-federation.org/resource/world-heart-report-2023/>.
- [4] A. O. Mocumbi, “Cardiovascular Health Care in Low- and Middle-Income Countries”, *Circulation*, vol. 149, no. 8, pp. 557–559, Feb. 2024. DOI: 10.1161/CIRCULATIONAHA.123.065717. [Online]. Available: <https://doi.org/10.1161/CIRCULATIONAHA.123.065717>.
- [5] L. Grinsztajn et al., “Why do tree-based models still outperform deep learning on typical tabular data?”, in *Advances in Neural Information Processing Systems*, vol. 35, Curran Associates, Inc., 2022, pp. 507–520. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/hash/0378c7692da36807bdec87ab043cdadc-Abstract-Datasets_and_Benchmarks.html.
- [6] S. Uddin et al., “Confirming the statistically significant superiority of tree-based machine learning algorithms over their counterparts for tabular data”, *PLOS ONE*, vol. 19, no. 4, e0301541, Apr. 2024. DOI: 10.1371/journal.pone.0301541. [Online]. Available: <https://doi.org/10.1371/journal.pone.0301541>.
- [7] S. M. Lundberg et al., “A unified approach to interpreting model predictions”, in *Advances in Neural Information Processing Systems*, vol. 30, Curran Associates, Inc., 2017, pp. 4765–4774. [Online]. Available: <https://proceedings.neurips.cc/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html>.
- [8] S. M. Lundberg et al., “From local explanations to global understanding with explainable AI for trees”, *Nature Machine Intelligence*, vol. 2, no. 1, pp. 56–67, 2020. DOI: 10.1038/s42256-019-0138-9. [Online]. Available: <https://doi.org/10.1038/s42256-019-0138-9>.
- [9] A. Janosi et al., *Heart disease*, UCI Machine Learning Repository, 1989. DOI: 10.24432/C52P4X. [Online]. Available: <https://doi.org/10.24432/C52P4X>.
- [10] R. C. Detrano et al., “International application of a new probability algorithm for the diagnosis of coronary artery disease”, *The American Journal of Cardiology*, vol. 64, no. 5, pp. 304–310, 1989. [Online]. Available: <https://api.semanticscholar.org/CorpusID:23545303>.
- [11] L. Breiman, “Random forests”, *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001. DOI: 10.1023/A:1010933404324. [Online]. Available: <https://doi.org/10.1023/A:1010933404324>.
- [12] R. E. Schapire, “A brief introduction to boosting”, in *Proceedings of the Sixteenth International Joint Conference on Artificial Intelligence (IJCAI-99)*, Stockholm, Sweden: Morgan Kaufmann, 1999, pp. 1401–1406.

- [13] T. Chen et al., “XGBoost: A scalable tree boosting system”, in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, San Francisco, CA, USA: ACM, Aug. 2016, pp. 785–794. DOI: 10.1145/2939672.2939785. [Online]. Available: <https://doi.org/10.1145/2939672.2939785>.
- [14] P. Shinde et al., “A Survey on Machine Learning Techniques for Heart Disease Prediction”, in, *SN Computer Science*, vol. 6, no. 4, p. 334, Apr. 2025, ISSN: 2661-8907. DOI: 10.1007/s42979-025-03860-2. [Online]. Available: <https://doi.org/10.1007/s42979-025-03860-2>.

8. Phụ lục

1. **Resources:** Các file code gợi ý (hint) và bài giải (solution) có thể được tải tại [đây](#). (**Lưu ý:** Sau khi hết deadline của phần bài này, ad mới copy các nội dung bài giải vào đường dẫn).

2. **Rubric:**

Mục	Kiến thức	Đánh giá
1.	- Hiểu bối cảnh bài toán chẩn đoán bệnh tim. - Nắm cấu trúc bộ Cleveland Heart Disease gồm 303 mẫu và 13 đặc trưng. - Hiểu ý nghĩa của các đặc trưng lâm sàng và cách nhị phân hóa target.	- Trình bày được nguồn và cấu trúc của dataset. - Giải thích được ý nghĩa cơ bản của các feature. - Phân biệt được target gốc 0-4 và target nhị phân 0/1.
2.	- Xử lý missing values và scaling. - Feature engineering. - One-hot encoding. - Mutual Information. - Feature selection bằng Decision Tree.	- Thiết lập preprocessing chỉ từ tập train. - Tránh data leakage. - Phân biệt FE+MI và DT-based feature selection. - Giải thích được mục đích của bốn biến thể dữ liệu.
3.	- Naive Bayes và KNN. - Decision Tree. - Random Forest. - AdaBoost. - Gradient Boosting. - XGBoost.	- Giải thích được sự khác nhau giữa mô hình không thuộc họ cây và cây. - Phân biệt Bagging và Boosting. - Giải thích được cơ chế Random Forest, AdaBoost, Gradient Boosting và XGBoost.
4.	- Train/validation/test split. - Stratified K-Fold Cross-Validation. - Hyperparameter tuning. - Accuracy và F1-score.	- Chọn hyperparameter chỉ bằng dữ liệu train. - Không sử dụng test để chọn mô hình. - Đọc được sự khác nhau giữa validation và test. - Nhận biết ảnh hưởng của sample size nhỏ tới metric.
5.	- Feature importance của Decision Tree/Random Forest. - SHAP và Shapley values. - TreeSHAP. - Global và local explanations.	- Phân biệt impurity-based importance với SHAP values. - Giải thích được global bar plot, beeswarm và waterfall plot. - Hiểu rằng SHAP giải thích hành vi của mô hình, không chứng minh quan hệ nhân quả.
6.	- Xây dựng ứng dụng bằng Streamlit. - Đóng gói preprocessing và model bằng Pipeline. - GridSearchCV cho deployment model. - Cloudflared tunnel.	- Ứng dụng nhận input và trả prediction probability. - Pipeline xử lý được missing values và categorical features. - Phân biệt mô hình deployment với mô hình dùng để báo cáo test metric.

3. **Q&A:** Bạn có thể đặt thêm câu hỏi về nội dung bài đọc trong group Facebook hỏi đáp tại [đây](#). Tất cả câu hỏi sẽ được trả lời trong vòng tối đa 4 tiếng.

AIO_Family

🔒 Private group · 2.0K members



Hình 25: Hình ảnh group facebook AIO Q&A.